Data Structures and Algorithms
Lecture 2 – Stacks

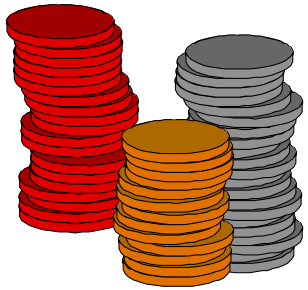
M.G.Noel A.S Fernando (PhD)

Professor/NSBM

Lecture 2

STACK

- A stack is a list of elements in which an element may be inserted or deleted only at one end, called the top of the stack.
- Stacks are sometimes known as LIFO (last in, first out) lists.



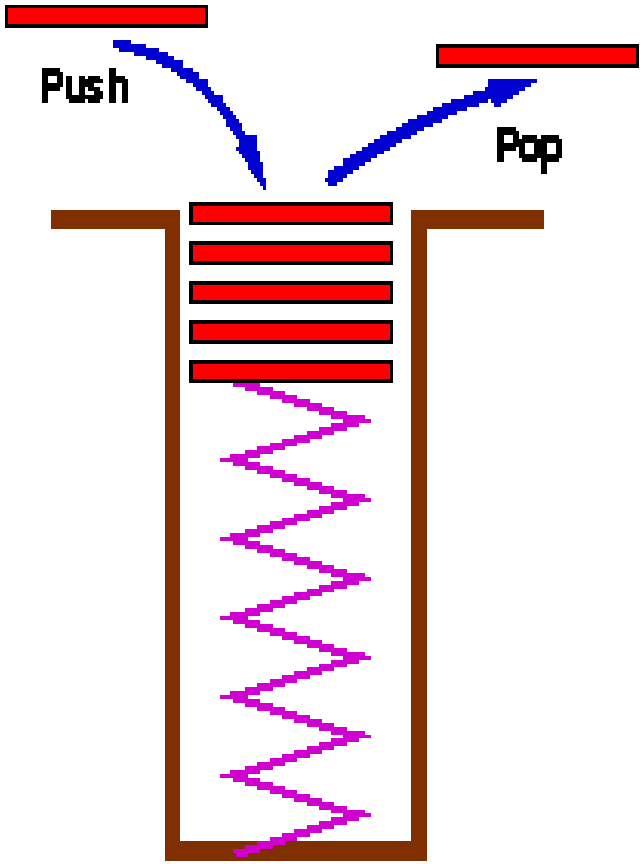
Stacks

- More examples from the real world?
- Stack of plates in a buffet table
- Stack of chairs
- The tennis balls in their container
- stack of trays in a criteria
- Stack of Books, Clothes piled

What is a stack?

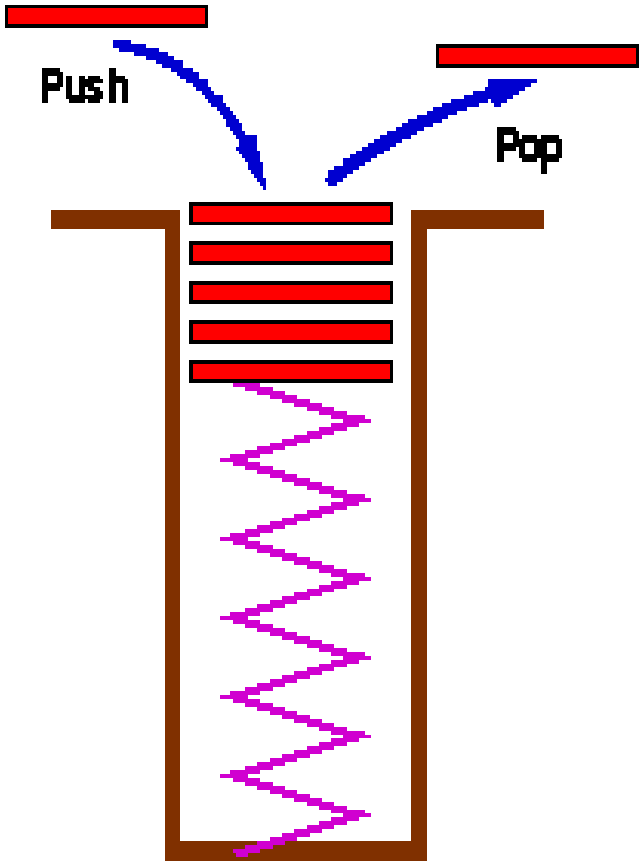
- Stores a set of elements in a particular order
- Stack principle: LAST IN FIRST OUT
- (LIFO)
- It means: the last element inserted is the first one to be removed.

Examples



- A common model of a stack is a plate or coin stacker. Plates are "pushed “onto to the top and "popped" off from the top.

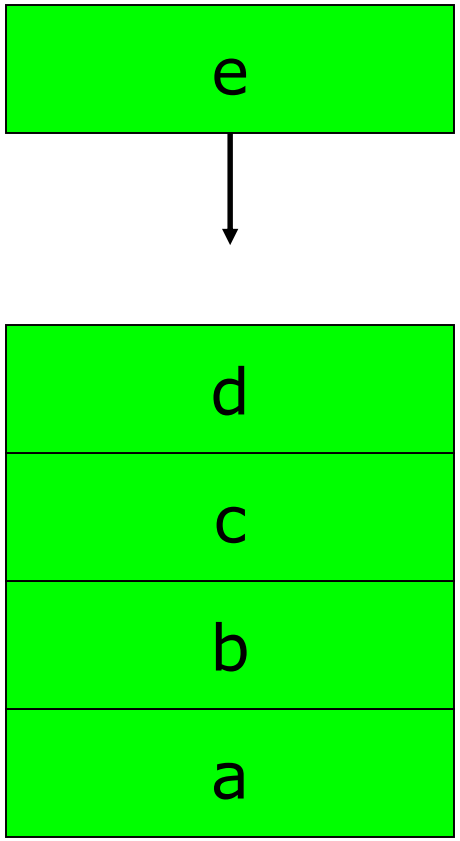
Examples (Cont.)



- Stacks form Last-In-First-Out (LIFO) queues.

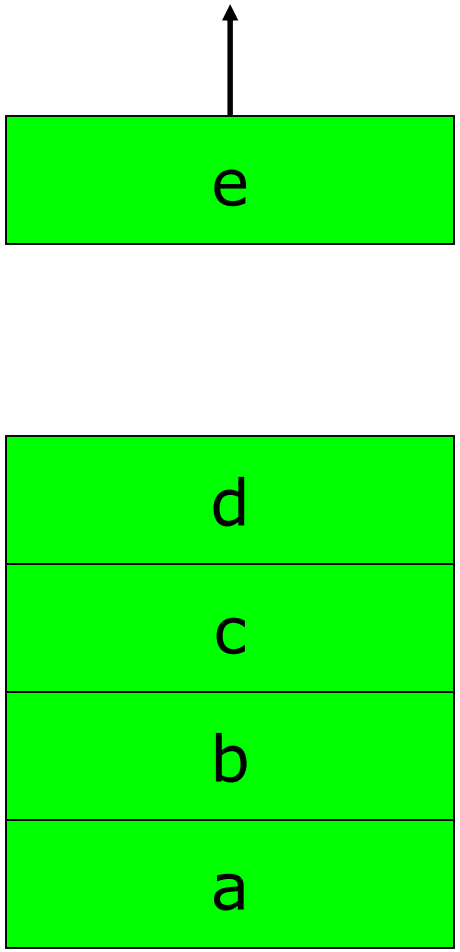
Stacks are LIFO

Push operations:



Stacks are LIFO

Pop operation:



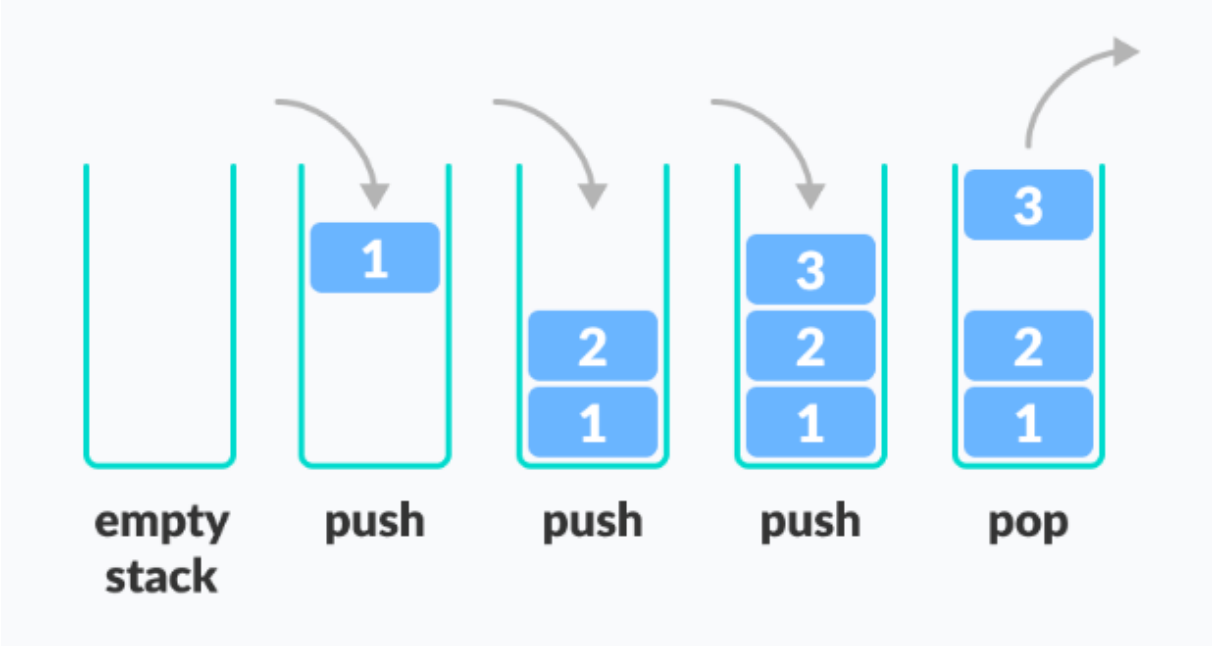
Last element that was pushed is the first to be popped.

Stacks

- *There are certain situations in computer science that one wants to restrict insertions and deletions so that they can take place only at the beginning or the end of the list, not in the middle*

Linear Data Structure

-E.g. Stack



Stack applications

- "Back" button of Web Browser
  - History of visited web pages is pushed onto the stack and popped when "back" button is clicked
- "Undo" functionality of a text editor
- Reversing the order of elements in an array
- Saving local variables when one function calls another, and this one calls another, and so on.

Further, Stacks application in Computing

- The ordinary stack is one of the most important data structures in all of computing.
  - function/method calls are placed onto a stack
  - compilers use stacks to evaluate expressions
  - stacks are great for reversing things, matching up related pairs of things, and backtracking algorithms
  - stack programming problems:
    - reverse letters in a string, reverse words in a line, or reverse a list of numbers
    - find out whether a string is a palindrome
    - examine a file to see if its braces { } and other operators match
    - convert infix expressions to postfix or prefix

Stacks

- A stack is a linear data structure that can be accessed only at one of its ends for storing and retrieving data.
- There are two ways of implementing a stack : Array (Static) and linked list (dynamic).

Abstract Data Types

- ADTs provide a way for us to formally define reusable modules in a way that is mathematically sound, precise, and unambiguous.
- Abstract Data Types are focused on what, not how (they're framed declaratively, and do not specify algorithms or data structures).
- Common examples include lists, stacks, sets, etc.

Basic operations on Stacks

- A stack is an object or more specifically an ADT that allows the following operations:

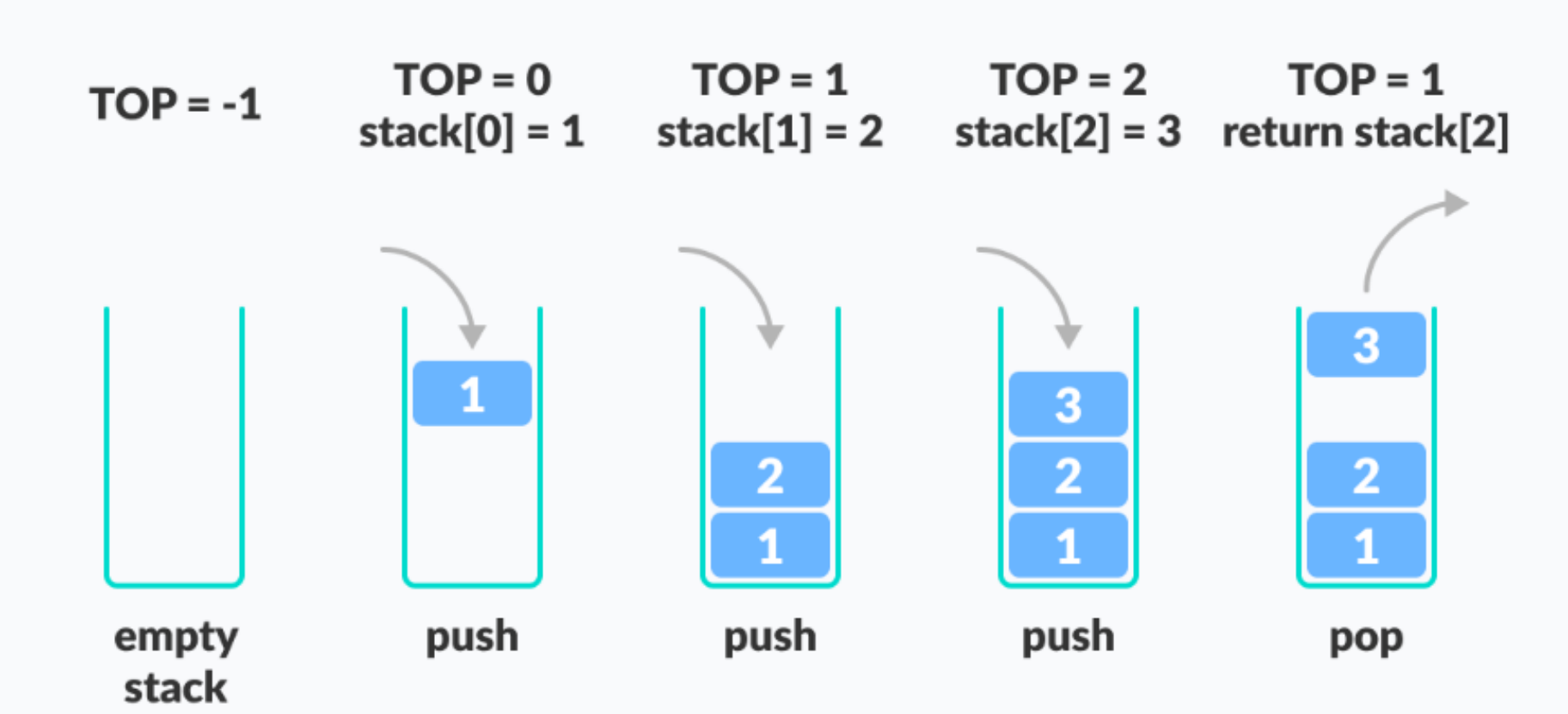
  - `Push` : Add an element to the top of a stack
  - `Pop` : Remove an element from the top of a stack
  - `IsEmpty` : Check if the stack is empty
  - `IsFull` : Check if the stack is full
  - `Peek` : Get the value of the top element without removing it

How a Stack Works

- The operations work as follows:

1. A pointer called `TOP` is used to keep track of the top element in the stack.
2. When initializing the stack, we set its value to -1 so that we can check if the stack is empty by comparing `TOP == -1`.
3. On pushing an element, we increase the value of `TOP` and place the new element in the position pointed to by `TOP`.
4. On popping an element, we return the element pointed to by `TOP` and reduce its value.
5. Before pushing, we check if the stack is already full
6. Before popping, we check if the stack is already empty

How a Stack Works



Stack Implementation

- Implementation can be done in two ways
  - Static implementation
  - Dynamic Implementation
- Static Implementation
  - Stacks have **fixed size**, and are implemented as **arrays**
  - It is also inefficient for utilization of memory
- Dynamic Implementation
  - Stack **grow in size** as needed, and implemented as **linked lists**
  - Dynamic Implementation is done through pointers
  - The memory is efficiently utilize with Dynamic Implementations

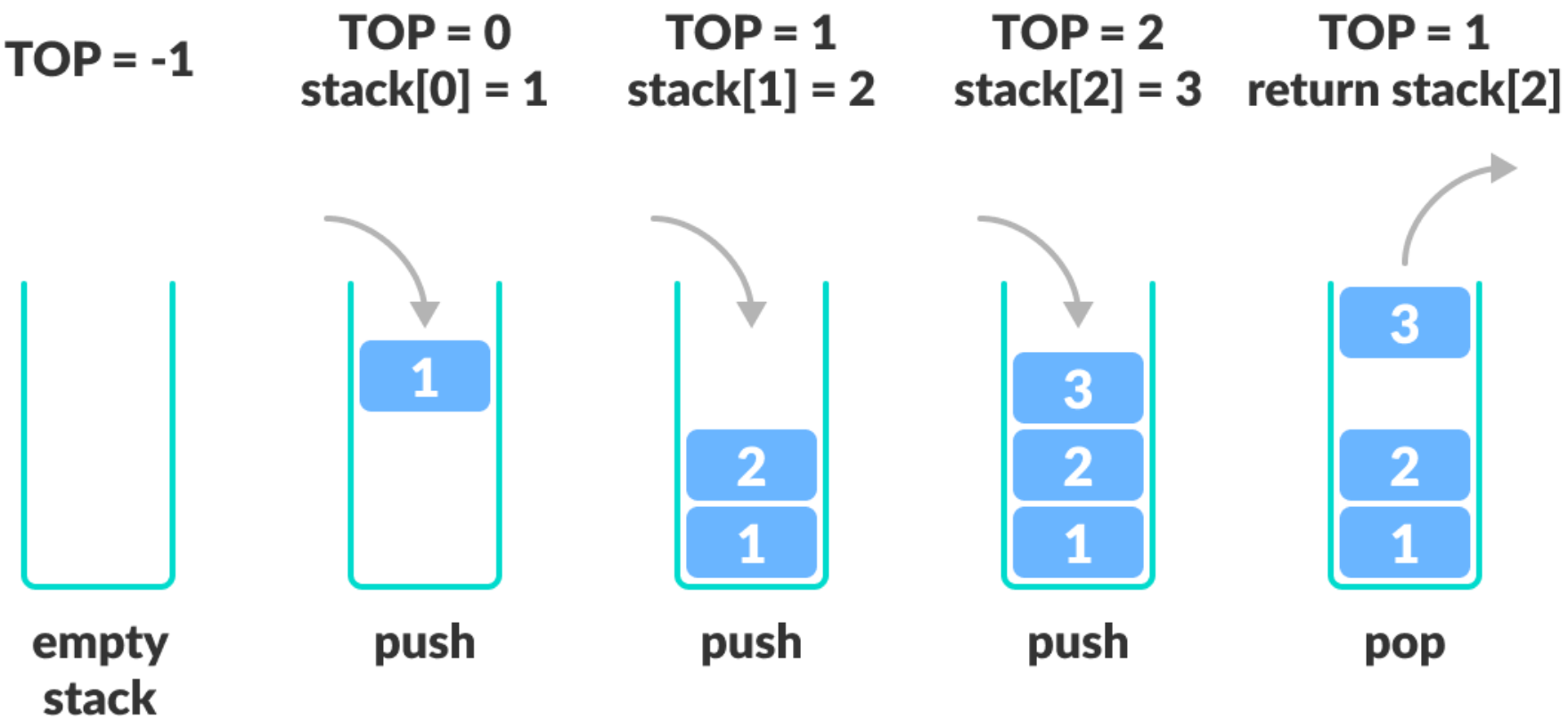
Stack-Related Terms

- Top
  - A pointer that points the top element in the stack.
- Stack Underflow
  - When there is no element in the stack, the status of stack is known as stack underflow.
- Stack Overflow
  - When the stack contains equal number of elements as per its capacity and no more elements can be added, the status of stack is known as stack overflow

Representation of Stack

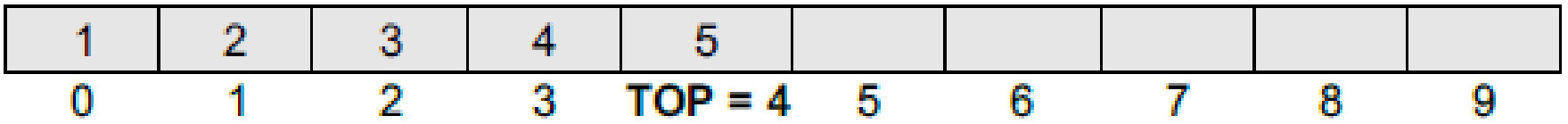
- Consider a stack with 6 elements capacity, This is called as the size of the stack.
- The number of elements to be added should not exceed the maximum size of the stack.
- If we attempt to add new element beyond the maximum size, we will encounter a *stack overflow* condition.
- Similarly, you cannot remove elements beyond the base of the stack. If such is the case, we will reach a *stack underflow* condition

Example : Push(Insert) operations on stack -push()

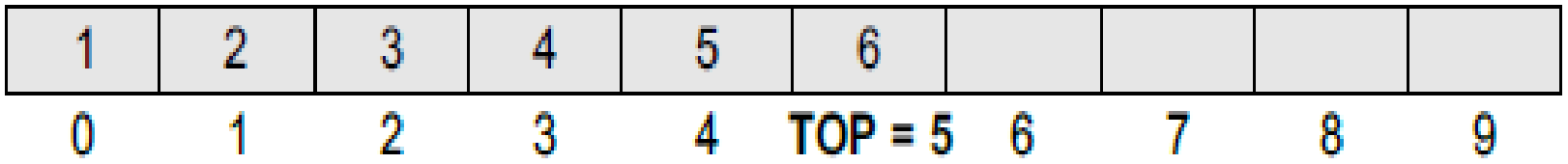


Algorithm to insert an element in a stack

E.g.1 Consider the following stack.



To insert an element with value 6, we first check if TOP=MAX–1. If the condition is false, then we increment the value of TOP and store the new element at the position given by stack[TOP].



Algorithm to insert an element in a stack

Example 2

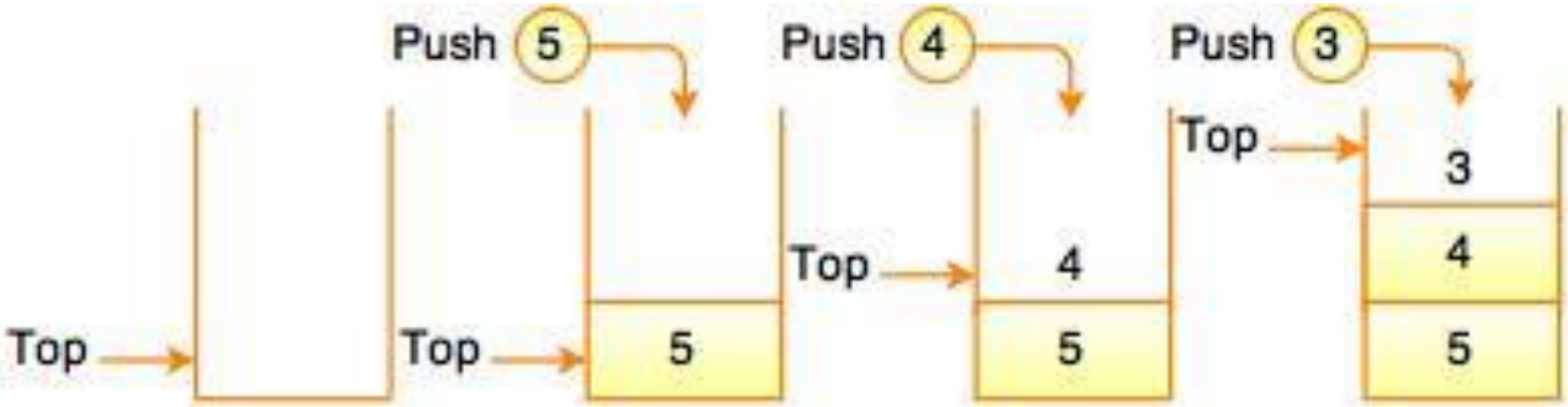


Fig. Insertion of Elements in a Stack

Question :Write a Pseudocode Algorithm to insert (push) an element into a stack

Algorithm to insert (push) an element in a stack

```
Step 1: IF TOP = MAX-1
            PRINT "OVERFLOW"
            Goto Step 4
        [END OF IF]
Step 2: SET TOP = TOP + 1
Step 3: SET STACK[TOP] = VALUE
Step 4: END
```

C code segment to insert (push) an element in a stack

```
void push()
{
    int x;
    if (top == SIZE - 1)
    {
        printf("\nOverflow!!");
    }
    else
    {
        printf("\nEnter the element to be added onto the stack: ");
        scanf("%d", &x);
        top = top + 1;
        inp_array[top] = x;
    }
}
```

Program segment
insert (push) using Java

```
// push elements to the top of stack
public void push(int x) {
if (isFull()) {
System.out.println("Stack OverFlow");
// terminates the program
System.exit(1);
// insert element on top of stack
System.out.println("Inserting " + x); arr[++top] = x;
}
```

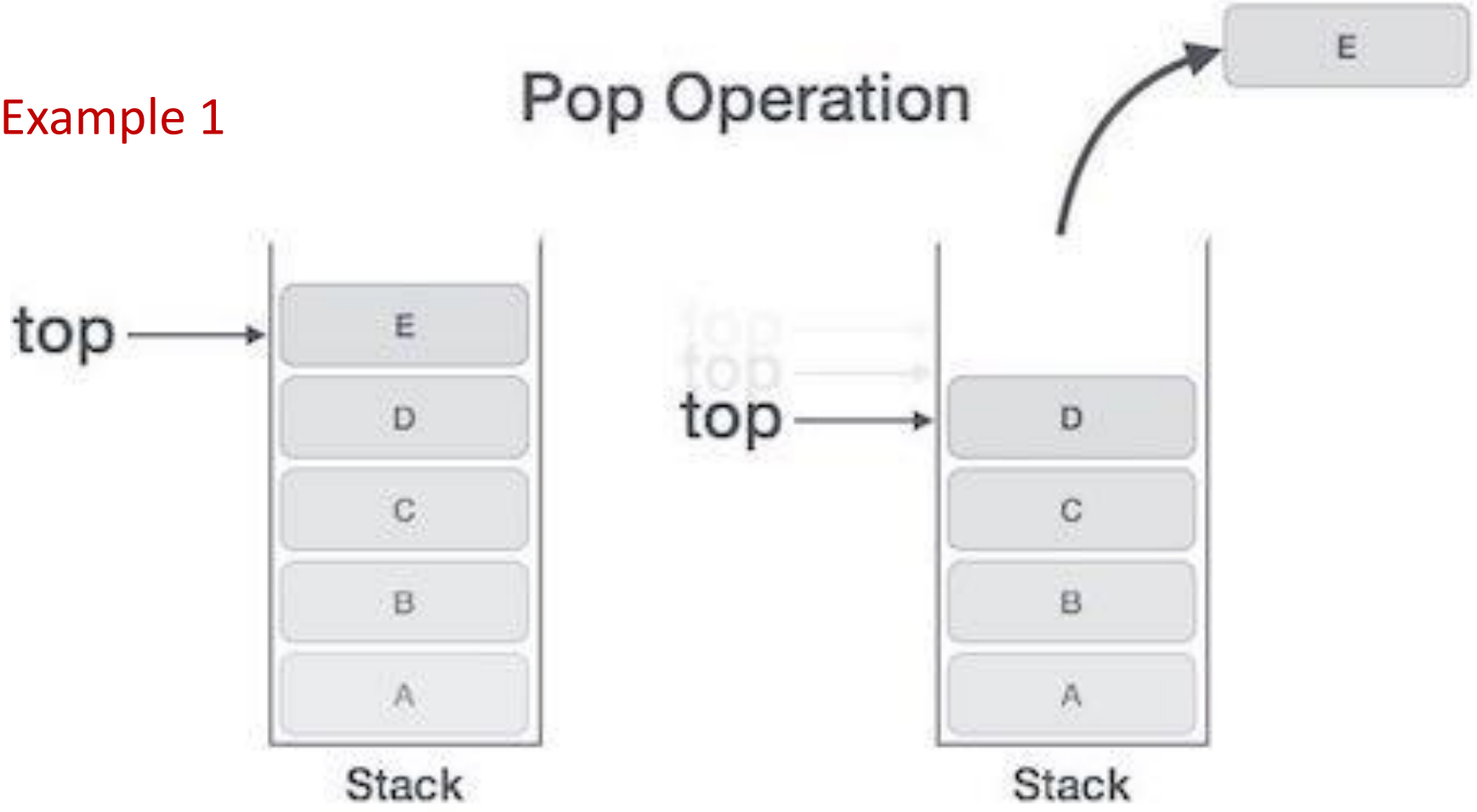
- Note : exit(1) indicates abnormal termination of the JVM

Pop (delete) Operation

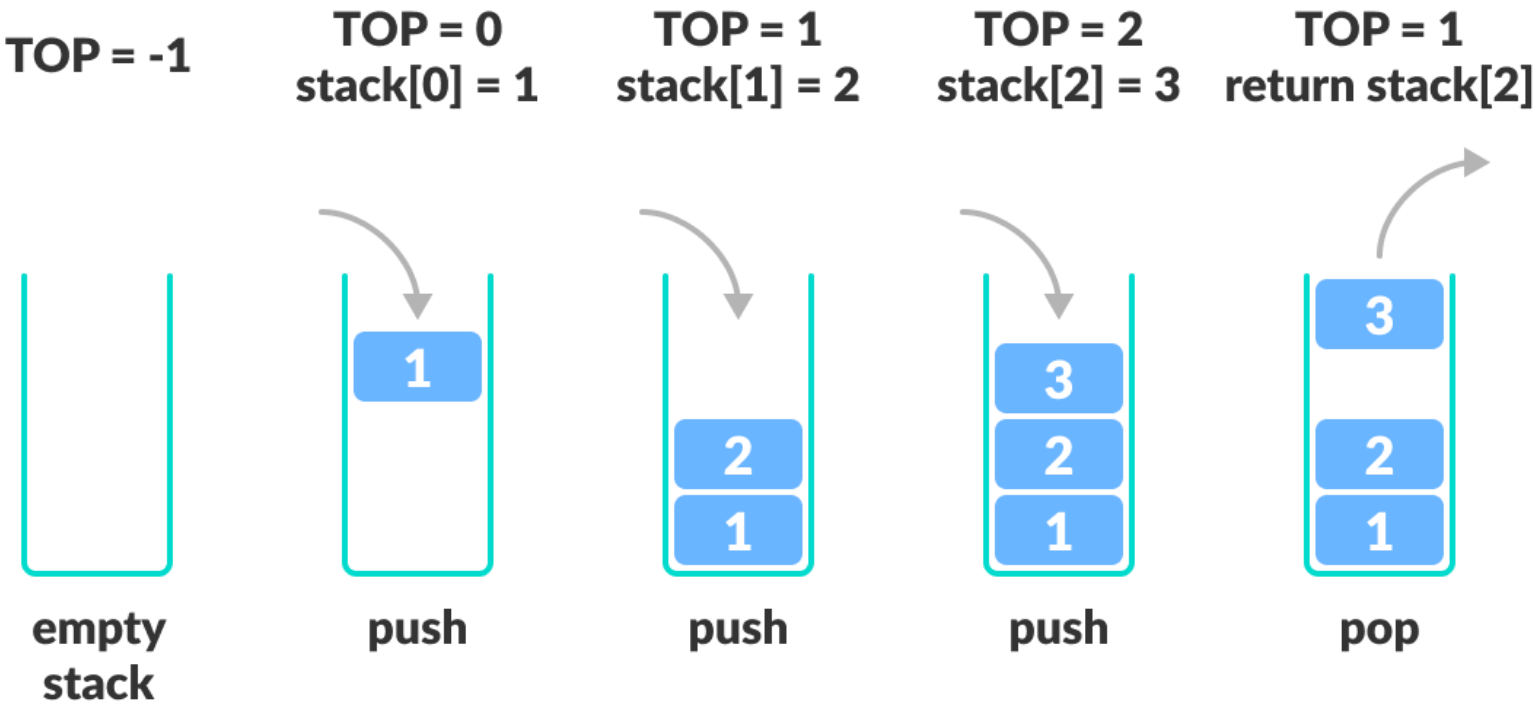
- The pop operation is used to delete the topmost element from the stack.
- However, before deleting the value, we must first check if TOP=NULL, because if stack is empty then no more deletions can be done.
- If an attempt is made to delete a value from a stack that is already empty, an UNDERFLOW message is printed.

Pop Operation

Example 1

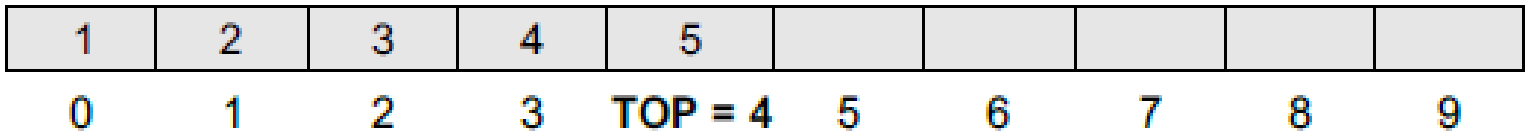


Pop Operation

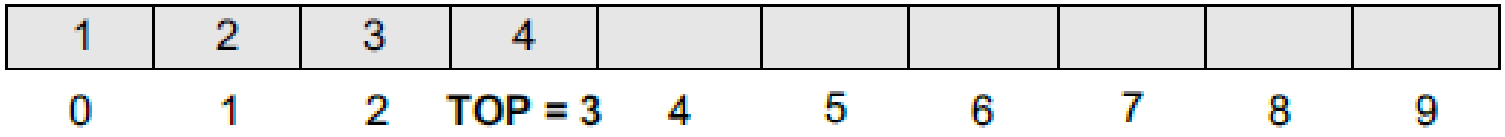


Pop Operation

- Example 2
- To delete the topmost element, we first check if TOP=NULL. If the condition is false, then we decrement the value pointed by TOP.

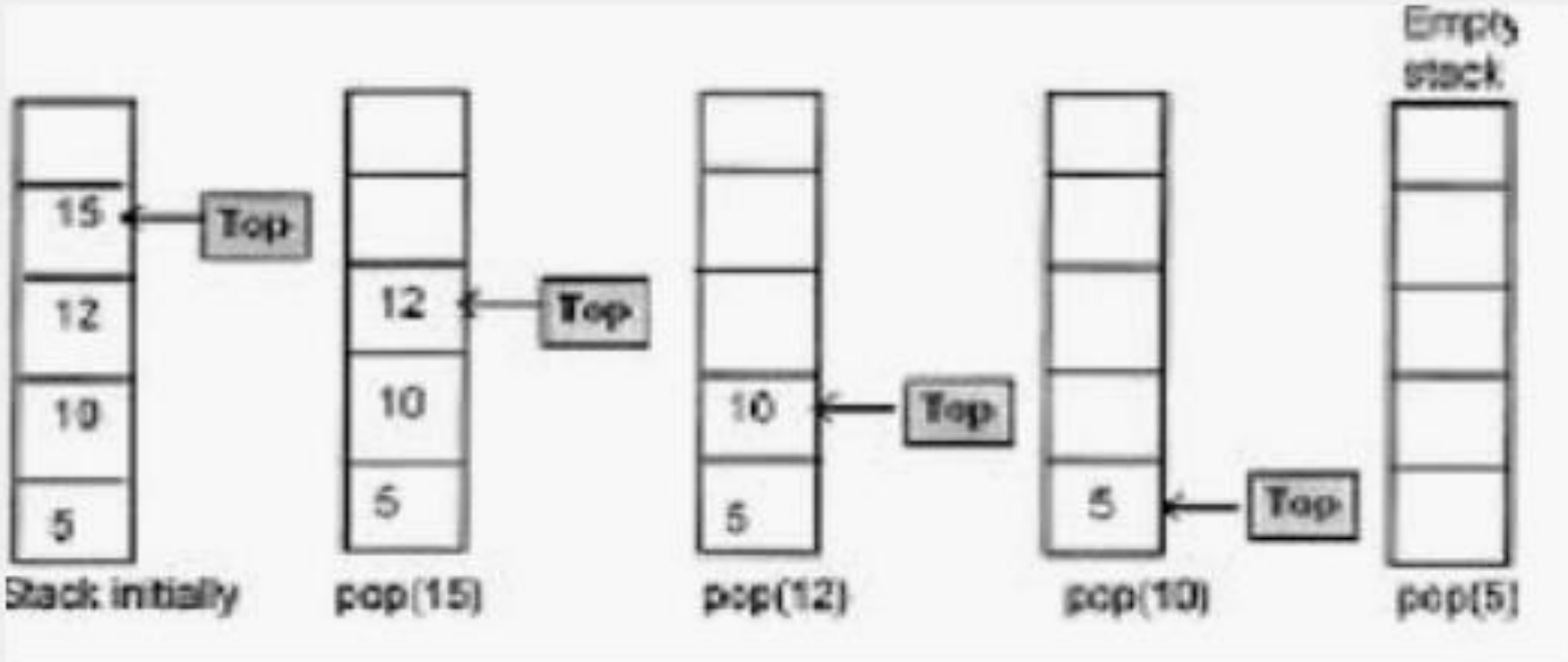


- Thus, the updated stack becomes as shown in below



Pop operations on stack

When an element is taken off from the stack, the operation is performed by pop().



Question: Write a Pseudo code / C code segment to pop (delete) an element from a stack

Question: Write a Pseudo code / C code segment to pop (delete) an element from a stack

```
Step 1: IF TOP = NULL
            PRINT "UNDERFLOW"
            Goto Step 4
        [END OF IF]
Step 2: SET VAL = STACK[TOP]
Step 3: SET TOP = TOP - 1
Step 4: END
```

Question: Write a Pseudo code / C code segment to pop (delete) an element from a stack

```
void pop()
{
    if (top == -1)
    {
        printf("\nUnderflow!!");
    }
    else
    {
        printf("\nPopped element: %d", inp_array[top]);
        top = top - 1;
    }
}
```

Program segment
pop (delete) using Java

```
// pop elements from top of stack
public int pop() {
    // if stack is empty
    // no element to pop
    if (isEmpty()) {
        System.out.println("STACK EMPTY");
        // terminates the program
        System.exit(1); }
        // pop element from top of stack
        return arr[top--];
 }
```

Peek Operation

- Peek is an operation that returns the value of the topmost element of the stack without deleting it from the stack

```
Step 1: IF TOP = NULL
            PRINT "STACK IS EMPTY"
            Goto Step 3
Step 2: RETURN STACK[TOP]
Step 3: END
```

C code segment for Peek Operation

```
/* Function to return the topmost element in the stack */
 int peek()
{
return stack[top];
}
```

Improvements

1. What is the boundary condition that can be used with Peek?
2. How can we enhance the above code by introducing the boundary condition?

Verifying whether the Stack is empty: isEmpty()

- The *isEmpty()* operation verifies whether the stack is empty.
- This operation is used to check the status of the stack with the help of top pointer.
- Pseudocode

1. START
2. If the top value is -1, the stack is empty. Return 1.
3. Otherwise, return 0.
4. END

isempty()-Pseudocode

isEmpty() – check if stack is empty

```
Begin Procedure IsEmpty
        If top is -1
                return Ture
        else
                return False
        endif
End Procedure
```

C Code segment for Isempty()

```
/* Check if the stack is empty */
 int isempty()
{
    if(top == -1)
        return 1;
     else
       return 0;
 }
```

Verifying whether the Stack is full: isFull()

- The *isFull()* operation checks whether the stack is full.
- This operation is used to check the status of the stack with the help of top pointer.

Pesudocode Algorithm of isfull() function

```
Begin Procedure IsFull
        If top is equal to capacity-1
                return true
        else
                return false
        endif
End procedure
```

Verifying whether the Stack is full: isFull()

- Pseudocode Algorithm

1. START
2. If the size of the stack is equal to the top position of the stack, the stack is full. Return 1.
3. Otherwise, return 0.
4. END

Verifying whether the Stack is full: isFull()

C Code Segment

```
/* Check if the stack is full */
 int isfull()
{
if(top == MAXSIZE)
return 1;
else
return 0;
 }
```

Program to perform Push, Pop, and Peek operations on a stack using C

This is only for knowledge enhancement, not considered for evaluation purposes.

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#define MAX 3 // Altering this value changes size of stack created

int st[MAX], top=-1;
void push(int st[], int val);
int pop(int st[]);
int peek(int st[]);
void display(int st[]);
```

Program to perform Push, Pop, and Peek operations on a stack using C

```
int main(int argc, char *argv[]) {
        int val, option;
        do
        {
                printf("\n *****MAIN MENU*****");
                printf("\n 1. PUSH");
                printf("\n 2. POP");
                printf("\n 3. PEEK");
                printf("\n 4. DISPLAY");
                printf("\n 5. EXIT");
                printf("\n Enter your option: ");
                scanf("%d", &option);
                switch(option)
```

Program to perform Push, Pop, and Peek operations on a stack using C

```
{
case 1:
        printf("\n Enter the number to be pushed on stack: ");
        scanf("%d", &val);
        push(st, val);
        break;
case 2:
        val = pop(st);
        if(val != -1)
        printf("\n The value deleted from stack is: %d", val);
        break;
case 3:
        val = peek(st);
        if(val != -1)
```

Program to perform Push, Pop, and Peek operations on a stack using C

```
                        printf("\n The value stored at top of stack is: %d", val);
                        break;
                case 4:
                        display(st);
                        break;
                }
        }while(option != 5);
        return 0;
}
void push(int st[], int val)
{
        if(top == MAX-1)
        {
                printf("\n STACK OVERFLOW");
        }
        else
        {
                top++;
                st[top] = val;
        }
}
```

Program to perform Push, Pop, and Peek operations on a stack using C

```
int pop(int st[])
{
        int val;
        if(top == -1)
        {
                printf("\n STACK UNDERFLOW");
                return -1;
        }
        else
        {
                val = st[top];
                top--;
                return val;
        }
}
```

Program to perform Push, Pop, and Peek operations on a stack using C

```
void display(int st[])
{
        int i;
        if(top == -1)
        printf("\n STACK IS EMPTY");
        else
        {
                for(i=top;i>=0;i--)
                printf("\n %d",st[i]);
                printf("\n"); // Added for formatting purposes
        }
}
int peek(int st[])
{
        if(top == -1)
        {
                printf("\n STACK IS EMPTY");
                return -1;
        }
        else
        return (st[top]);
}
```

•A stack is generally implemented with only two principle operations (apart from a constructor and destructor methods):

Push :adds an item to a stack

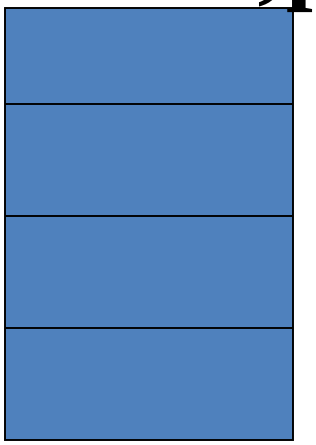
Pop :extracts the most recently pushed item from the stack

Other methods such as

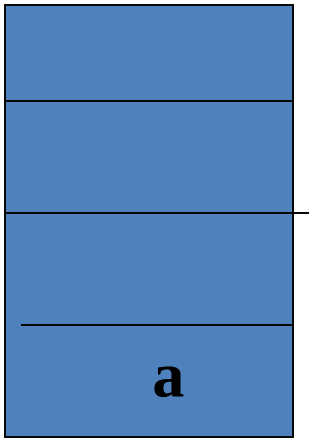
top: returns the item at the top *without removing it*

isempty :determines whether the stack has anything in it

How the stack routines work:empty stack;push(a), push(b); pop

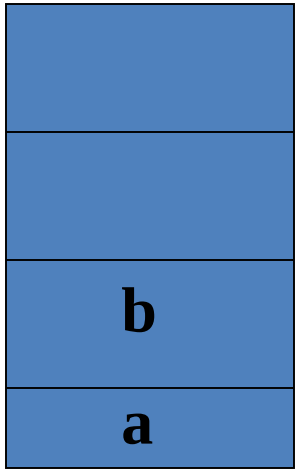


Stack is empty tos=-1



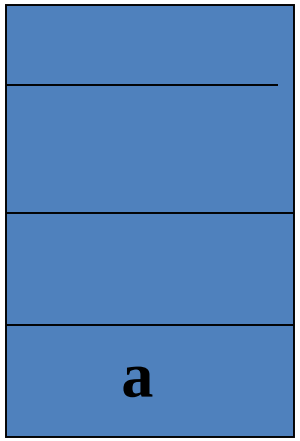
tos= 0

Push (a)



tos= 1

Push (b)



tos =0

pop

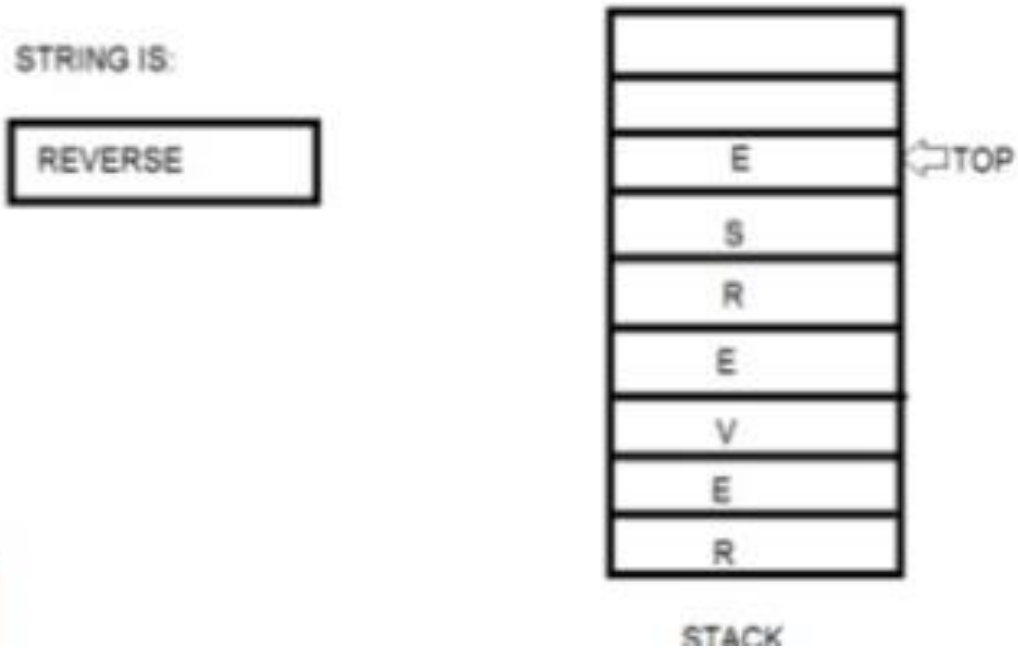
APPLICATIONS OF STACKS

- Reversing a list
- Parentheses checker
- Conversion of an infix expression into a postfix expression
- Evaluation of a postfix expression
- Conversion of an infix expression into a prefix expression
- Evaluation of a prefix expression
- Recursion
- Tower of Hanoi

Reversing a List

- A list of numbers/characters can be reversed by reading each number/character from an array starting from the first index and pushing it on a stack.
- Once all the numbers have been read, the numbers can be popped one at a time and then stored in the array starting from the first index.

To reverse the string 'REVERSE' the string is read from left to right and its characters are pushed . LIKE:



Reversing a List cont….

```
#include <stdio.h>
#include <conio.h>
int stk[10];
int top=-1;
int pop();
void push(int);
int main()
{
        int val, n, i,
        arr[10];
        clrscr();
        printf("\n Enter the number of elements in the array : ");
        scanf("%d", &n);
        printf("\n Enter the elements of the array : ");
        for(i=0;i<n;i++)
                scanf("%d", &arr[i]);
        for(i=0;i<n;i++)
                push(arr[i]);
        for(i=0;i<n;i++)
        {
                val = pop();
                arr[i] = val;
        }
```

Reversing a List cont….

```
        printf("\n The reversed array is : ");
        for(i=0;i<n;i++)
                printf("\n %d", arr[i]);
        getche"();
        return 0;
}
void push(int val)
{
        stk[++top] = val;
}
int pop()
{
        return(stk[top--]);
}
```

Reversing a List cont.…

Output

```
Enter the number of elements in the array : 5
Enter the elements of the array : 1 2 3 4 5
The reversed array is : 5 4 3 2 1
```

Balanced Parentheses

For example:

VALID INPUTS	INVALID INPUTS
{ } ({ [ ] }) { [ ] ( ) } [{ ({ } [ ] ({ })) }]	{ (} ([(( )]) { } [ ]) [{) } ( ] }]

containing nested parenthesis:

- Stacks are also used to check whether a given arithmetic expressions containing nested parenthesis is properly parenthesized.
- The program for checking the validity of an expression verifies that for each left parenthesis braces or bracket ,there is a corresponding closing symbol and symbols are appropriately nested.

Check for Balanced Brackets in an expression using Stack

- Given an expression string exp, write a program to examine whether the pairs and the orders of “{“, “}”, “(“, “)”, “[“, “]” are correct in expression .

Example:

Input: *exp = “[()]{}{[()()]()}”*

Output: *Balanced*

Input: *exp = “[(])”*

Output: *Not Balanced*

Balanced Parentheses

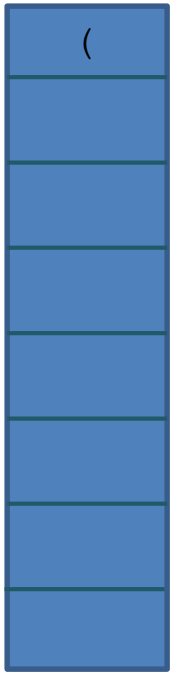
- When analyzing arithmetic expressions, it is important to determine whether an expression is balanced with respect to parentheses

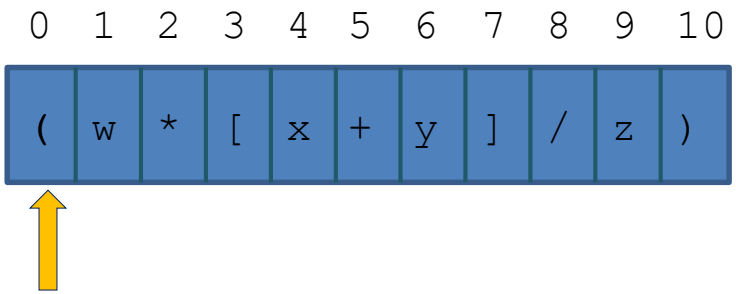
```
( a + b * ( c / ( d – e ) ) ) + ( d / e )
```

- The problem is further complicated if braces or brackets are used in conjunction with parentheses
- The solution is to use stacks!

Balanced Parentheses (cont.)

Expression: `(w * [x + y] / z)`

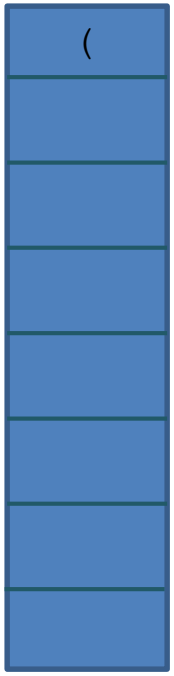


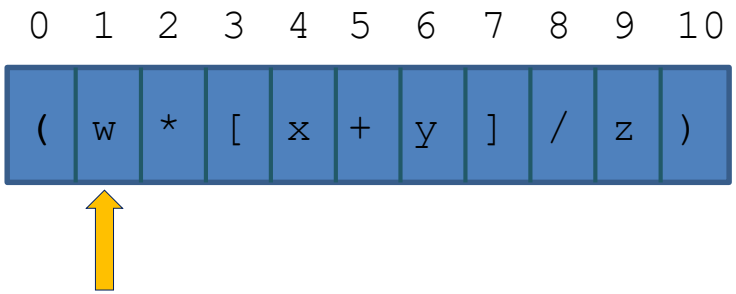



```
balanced : true
index    : 0
```

Balanced Parentheses (cont.)

Expression: `(w * [x + y] / z)`

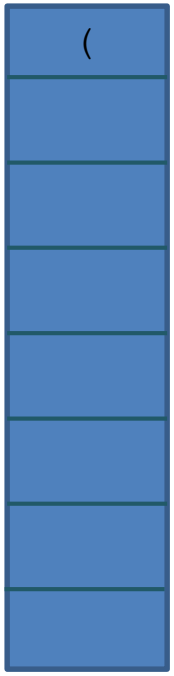


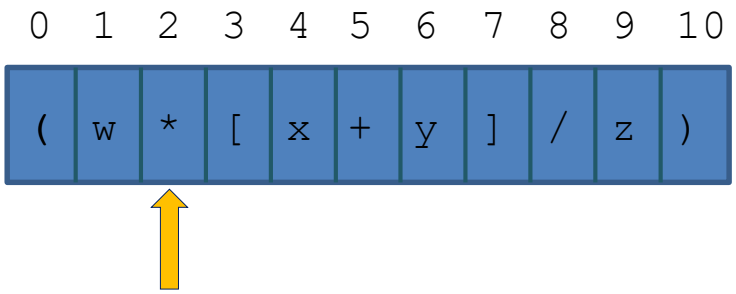



```
balanced : true
index    : 1
```

Balanced Parentheses (cont.)

Expression: `(w * [x + y] / z)`

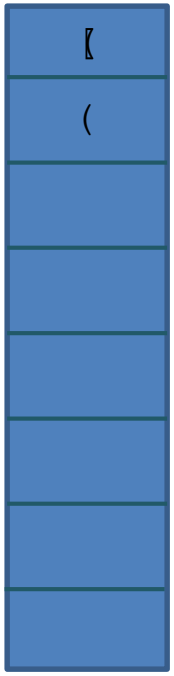


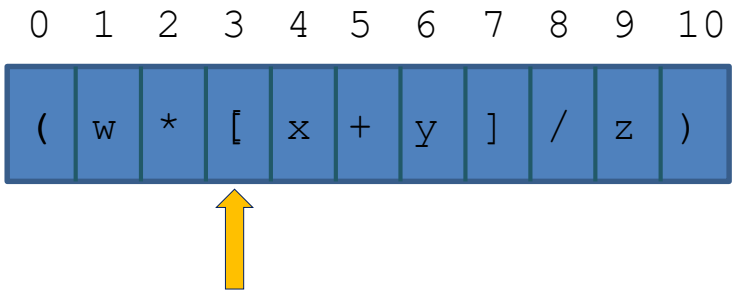



```
balanced : true
index    : 2
```

Balanced Parentheses (cont.)

Expression: `(w * [x + y] / z)`

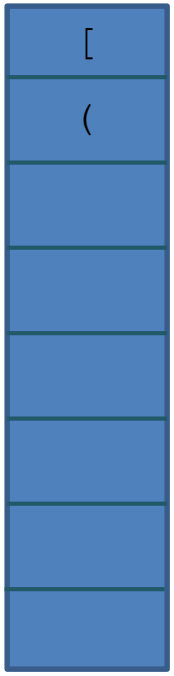


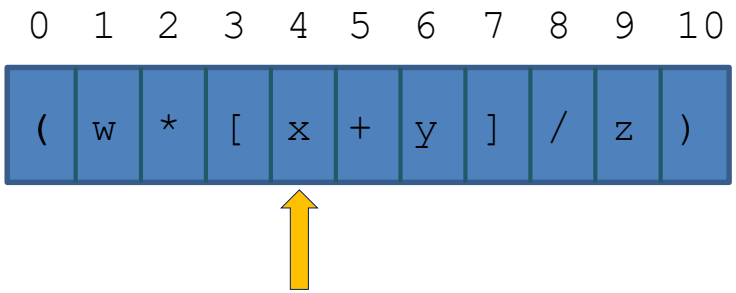



```
balanced : true
index    : 3
```

Balanced Parentheses (cont.)

Expression: `(w * [x + y] / z)`

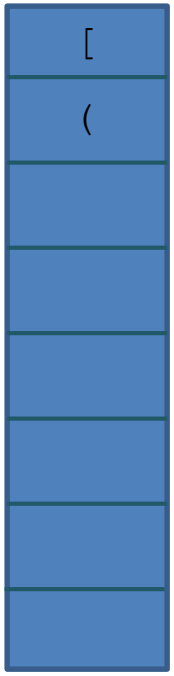


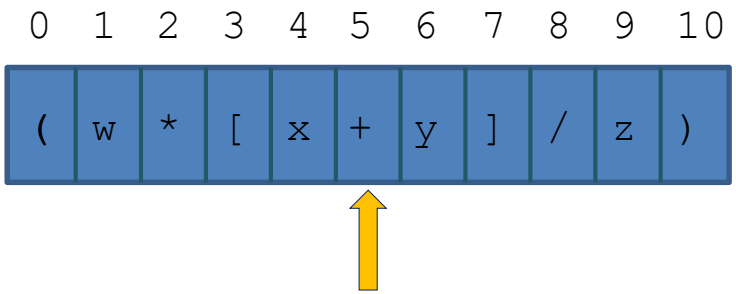



```
balanced : true
index    : 4
```

Balanced Parentheses (cont.)

Expression: `(w * [x + y] / z)`

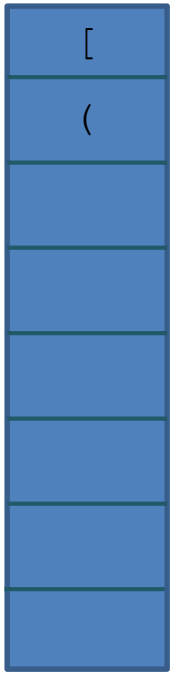


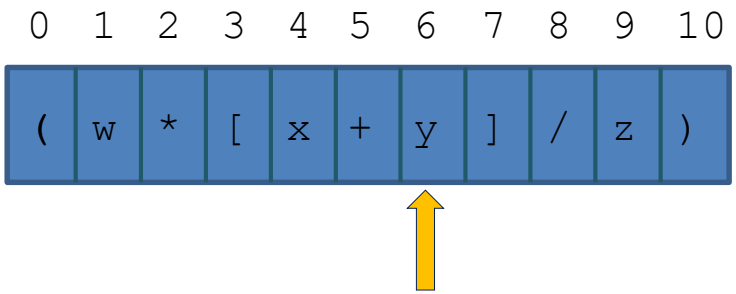



```
balanced : true
index    : 5
```

Balanced Parentheses (cont.)

Expression: `(w * [x + y] / z)`

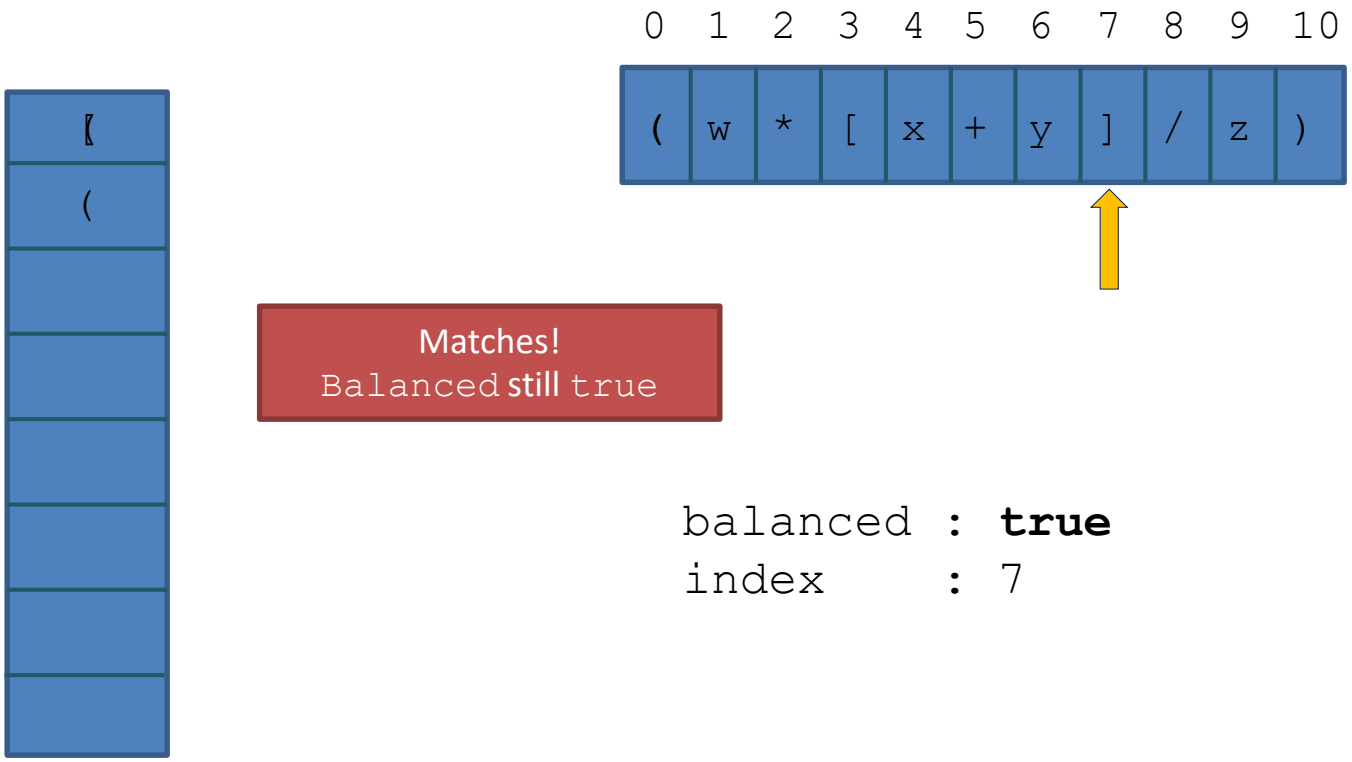





```
balanced : true
index    : 6
```

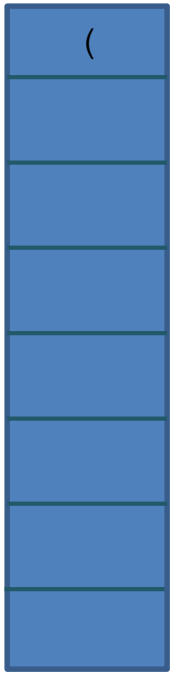
Balanced Parentheses (cont.)

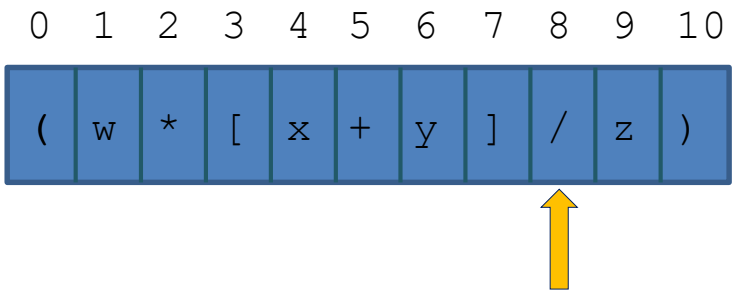
Expression: `(w * [x + y] / z)`



Balanced Parentheses (cont.)

Expression: `(w * [x + y] / z)`

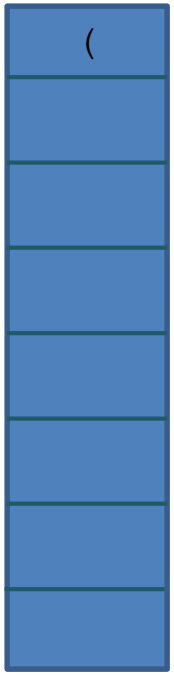


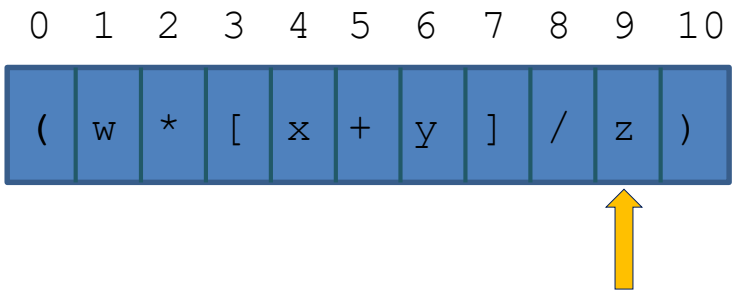



```
balanced : true
index    : 8
```

Balanced Parentheses (cont.)

Expression: `(w * [x + y] / z)`

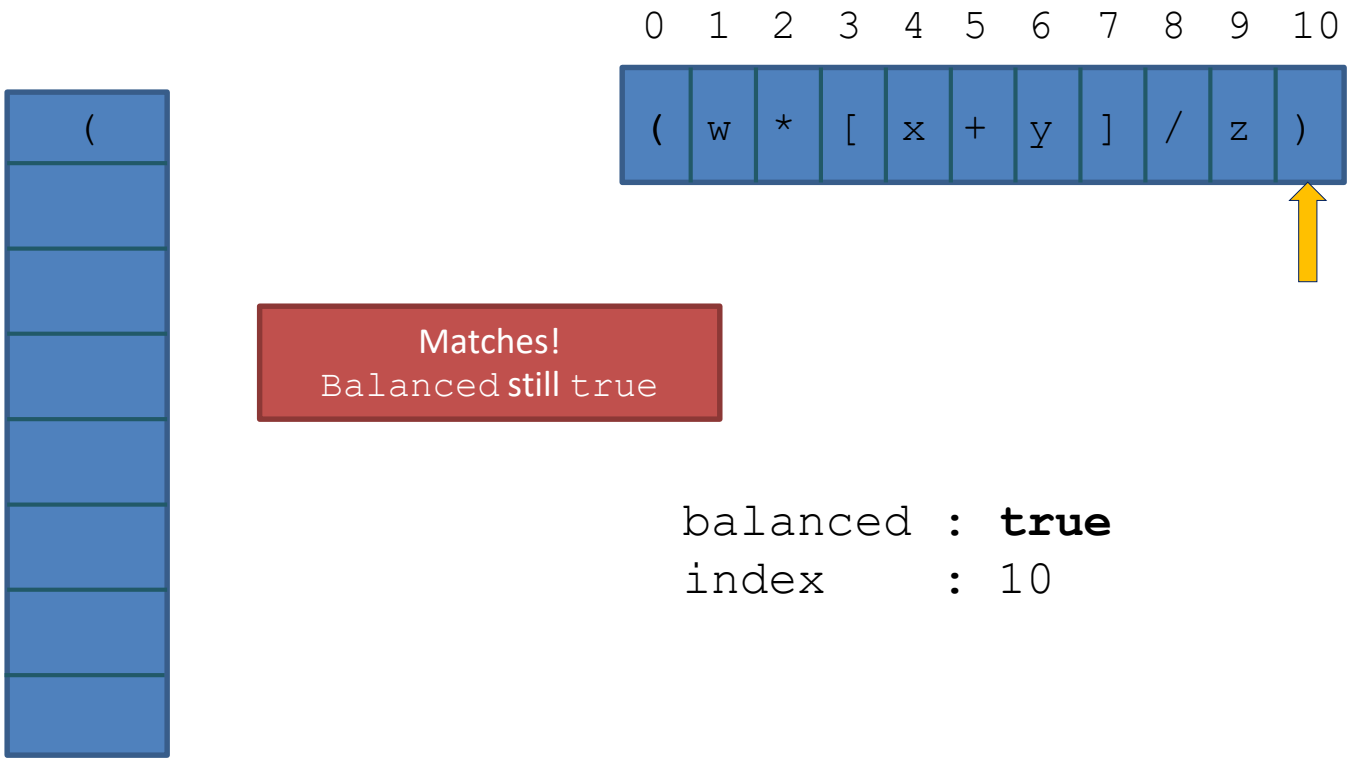





```
balanced : true
index    : 9
```

Balanced Parentheses (cont.)

Expression: `(w * [x + y] / z)`




```
balanced : true
index    : 10
```

Balanced Parentheses (cont.)

- How to use a stack to check for balanced symbols?

Note :Compilers check your program for syntax errors. However, frequently a lack of one symbol will cause the compiler to spill out a hundred lines of diagnostic without identifying the real error.

- For Example the sequence [()] is legal, but [(]) is wrong.
- Basic Algorithm

1. Make an empty stack
2. Read symbols until the end of the file
   a) If symbol is opening symbol, push it into the stack
   b) If it is closing symbol and if the stack is empty, then report an error.

c Otherwise, pop the stack, if the symbol popped is not in the corresponding opening symbol , then report an error

3. At the end of the file, if the stack is not empty, report an error.

Note : we should not consider a parenthesis as a symbol if it occurs inside a comment, string, constant or character string.

Home work- 2.6 Write a program to check nesting of parentheses using a stack.

- https://www.geeksforgeeks.org/check-for-balanced-parentheses-in-an-expression/

Solution

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
#define MAX 10
int top = -1;
int stk[MAX];
void push(char);
```

Solution cont....

```
char pop();
void main()
{
        char exp[MAX],temp;
        int i, flag=1;
        clrscr();
        printf("Enter an expression : ");
        gets(exp);
        for(i=0;i<strlen(exp);i++)
        {
                if(exp[i]=='(' || exp[i]=='{' || exp[i]=='[')
                        push(exp[i]);
                if(exp[i]==')' || exp[i]=='}' || exp[i]==']')
                        if(top == -1)
                                flag=0;
```

Solution cont.…

```
                else
                {
                        temp=pop();
                        if(exp[i]==')' && (temp=='{' || temp=='['))
                                flag=0;
                        if(exp[i]=='}' && (temp=='(' || temp=='['))
                                flag=0;
                        if(exp[i]==']' && (temp=='(' || temp=='{'))
                                flag=0;
                }
}
if(top>=0)
        flag=0;
if(flag==1)
        printf("\n Valid expression");
else
        printf("\n Invalid expression");
```

Solution cont.…

```
}
void push(char c)
{
        if(top == (MAX-1))
                printf("Stack Overflow\n");
        else
        {
                top=top+1;
                stk[top] = c;
        }
}
char pop()
{
        if(top == -1)
                printf("\n Stack Underflow");
        else
                return(stk[top--]);
}
```

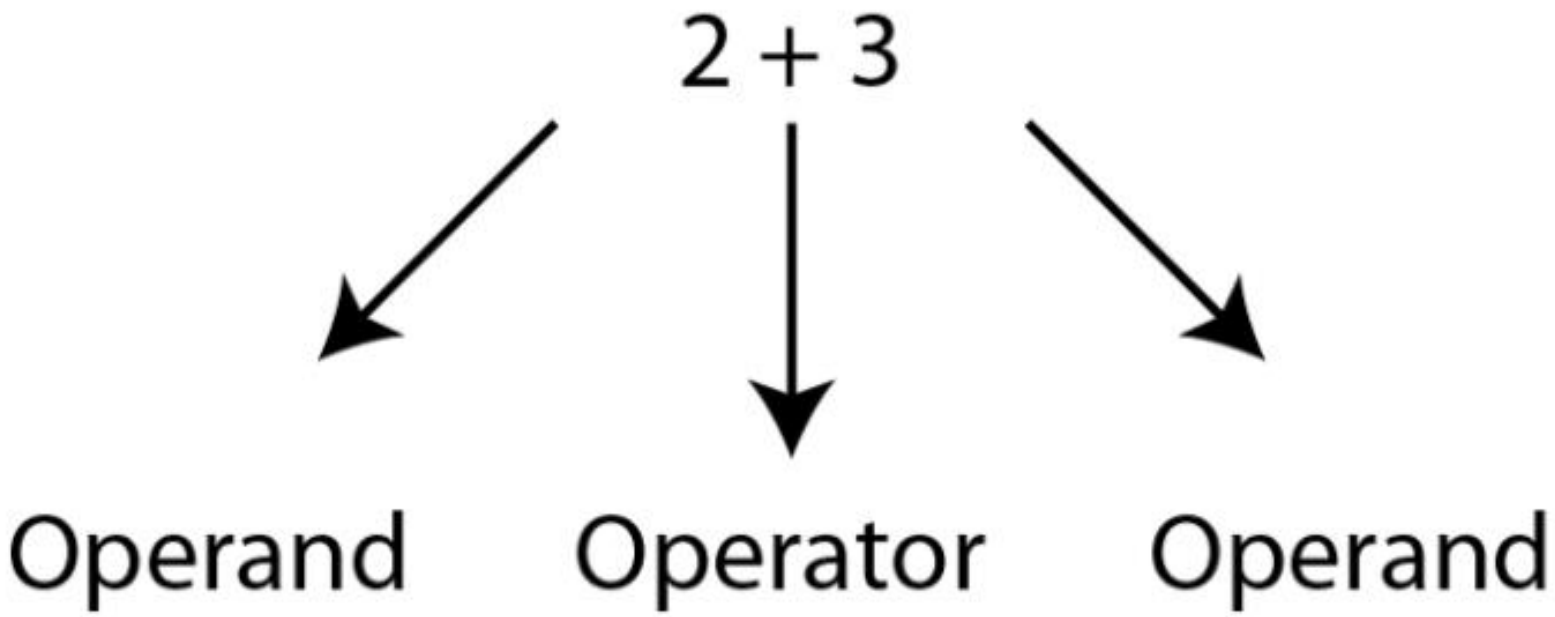
Output

```
Enter an expression : (A + (B - C))
Valid Expression
```

Evaluation of Arithmetic Expressions

$2 + 3$

and name the parts of the formulae as follows:



Evaluation of Arithmetic Expressions

- Infix, Postfix and Prefix
- Consider the sum of A and B, we think of applying the operator " + " to the operands A and B and write the sum as A+B. This particular representation is called **infix**.
- There are two alternative notations for expressing the sum of A and B using the symbols A,B and + These are

  +AB **Prefix**

  AB+ **Postfix**

- Suppose that we would now like to rewrite A+B*C in postfix, Appling the rules precedence, we obtain

A+(B*C) –Parentheses for emphasis

A+(BC*) – Convert the multiplication

A(BC*)+ -convert the addition

ABC*+ - postfix form

Eq: Infix -A+(B*C) = Postfix ABC*+

Infix –(A+B)*C = Postfix AB+C*

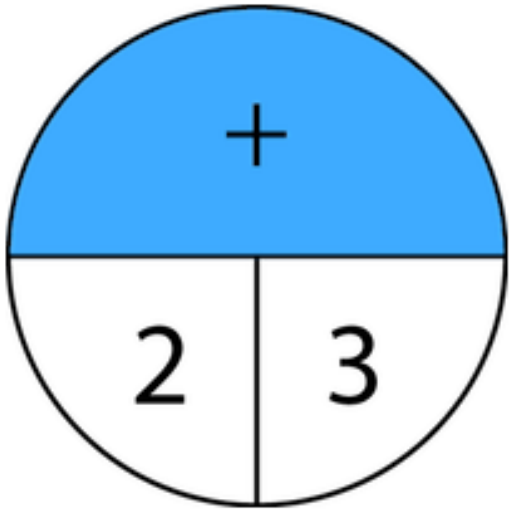
Question : How to evaluate a mathematical expression using a stack

Evaluating a postfix expression

- Initialise an empty stack
- While token remain in the input stream
  - Read next token
  - If token is a number, push it into the stack
  - Else, if token is an operator, pop top two tokens off the stack,apply the operator, and push the answer back into the stack
- Pop the answer off the stack.

Convert Infix expression (e.g., 2+3) into infix (e.g.,+23)

Which is then followed by the `operands`



Prefix is +23

Polish notation and Reverse Polish notation

- The two best known alternatives are where you write the operator before or after its operands - known as prefix or postfix notation.
- Polish logician Jan Łukasiewicz, invented (prefix) Polish notation in the 1920s - hence it is only natural that postfix notation is generally referred to as Reverse Polish Notation or RPN.

Reverse Polish Notation (RPN)

- Reverse Polish Notation (RPN): A mathematical notation in which operators follow operands.
- **Reverse Polish notation** (**RPN**), also known as **Polish postfix notation** or simply **postfix notation**

Note :BODMAS

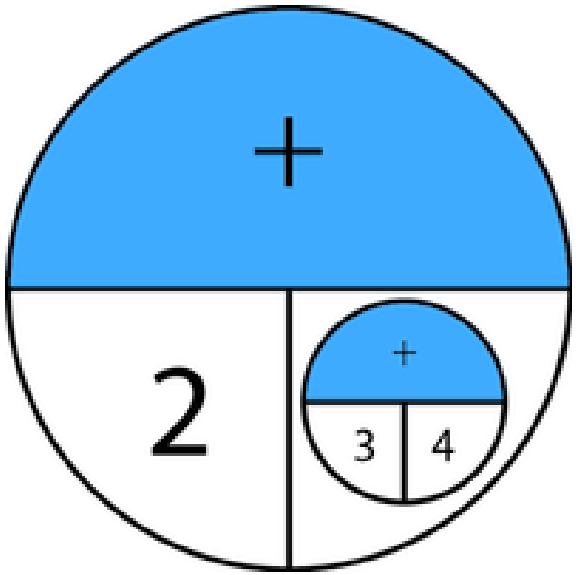
BODMAS tells us which order to perform a calculation in. Brackets first then Other then Division then Multiplication then Addition and finally Subtraction.

Convert 2 + (3 + 4) to postfix

Similarly when we have two expressions we have to remember the `operators` to append to the end.

2+(34+)

234++



Evaluate- 623+-382/+*2^3+

Symbol	opnd1	opnd2	value	opndstk
6				6
2				6,2
3				6,2,3
+	2	3	5	6,5
-	6	5	1	1
3	6	5	1	1,3

Evaluate- 623+-382/+*2^3+

Symbol	opnd1	opnd2	value	opndstk
8	6	5	1	1,3,8
2	6	5	1	1,3,8,2
/	8	2	4	1,3,4
+	3	4	7	1,7
*	1	7	7	7
2	1	7	7	7,2
^	7	2	49	49
3	7	2	49	49,3
+	49	3	52	52

Convert the following infix expressions into postfix expressions

a) **(A–B) * (C+D)**

b) **(A + B) / (C + D) – (D * E)**

c) **(A + B) * C**

Solution

(a) (A-B) * (C+D)

[AB-] * [CD+]

AB-CD+*

(b) (A + B) / (C + D) - (D * E)

[AB+] / [CD+] - [DE*]

[AB+CD+/] - [DE*]

AB+CD+/DE*-

Convert the following infix expressions into prefix expressions

a) **(A + B) * C**
b) **(A–B) * (C+D)**
c) **(A + B) / (C + D) – (D * E)**

Solution

(a) (A + B) * C

(+AB)*C

*+ABC

(b) (A-B) * (C+D)

[-AB] * [+CD]

*-AB+CD

(c) (A + B) / (C + D) - (D * E)

[+AB] / [+CD] - [*DE]

[/+AB+CD] - [*DE]

-/+AB+CD*DE

Exercise

- Convert the following infix expression into postfix expression using the algorithm given in above

- A – (B / C + (D % E * F) / G)* H)

Homework- 2.7

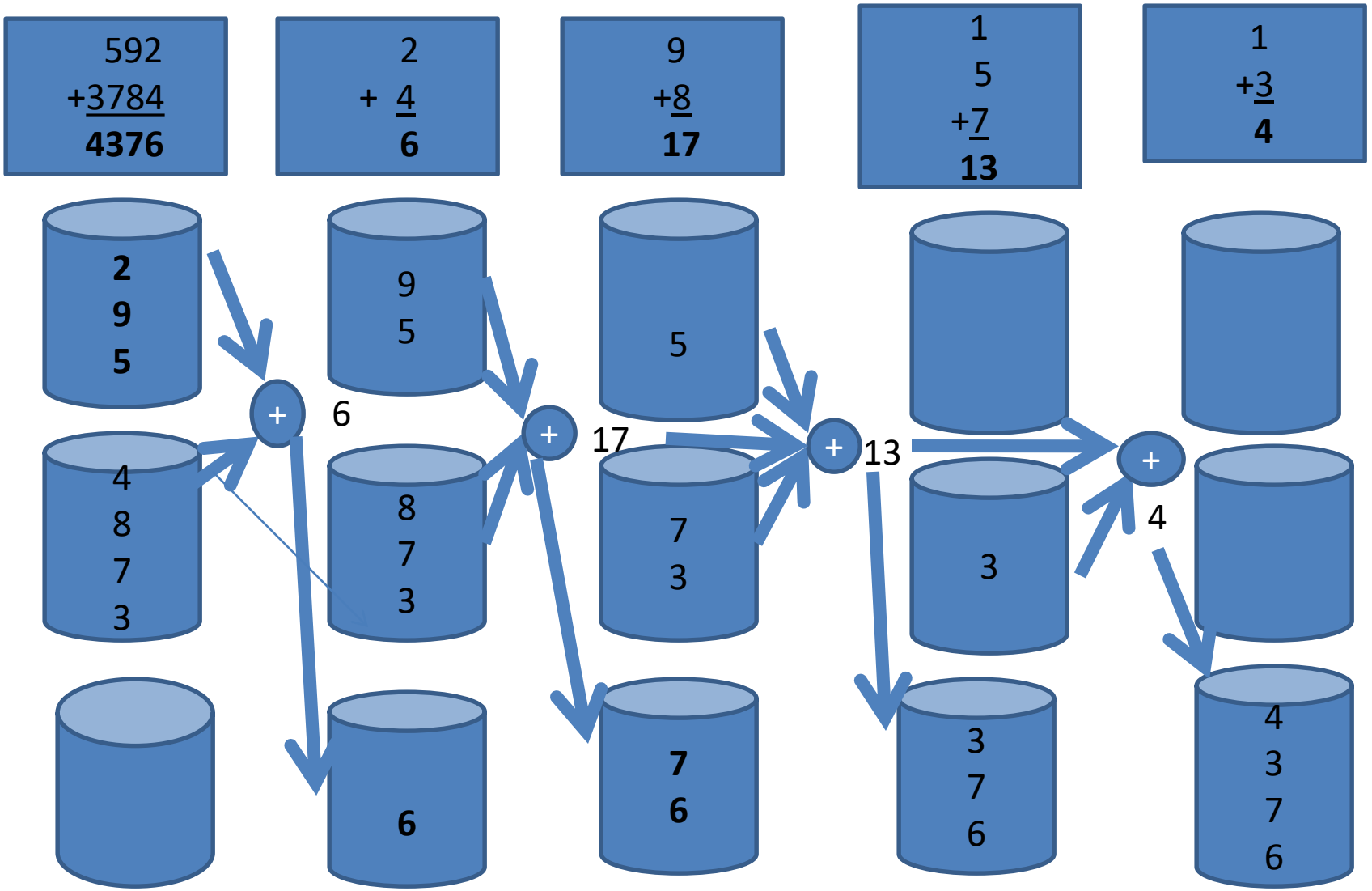
1. Write a program to convert an infix expression into its equivalent postfix notation.
2. **Converting expressions using Stack**

- Infix to prefix
- Postfix to infix
- Postfix to prefix
- Prefix to infix
- Prefix to postfix and coding

ANOTHER STACK APPLICATION

- Describe, how to add 592 and 3784 using Stacks

An example of adding 592 and 3784 using Stacks



STACK APPLICATIONS

- Write a pseudo code Algorithm to convert Decimal to Binary using a Stack

Stack Applications

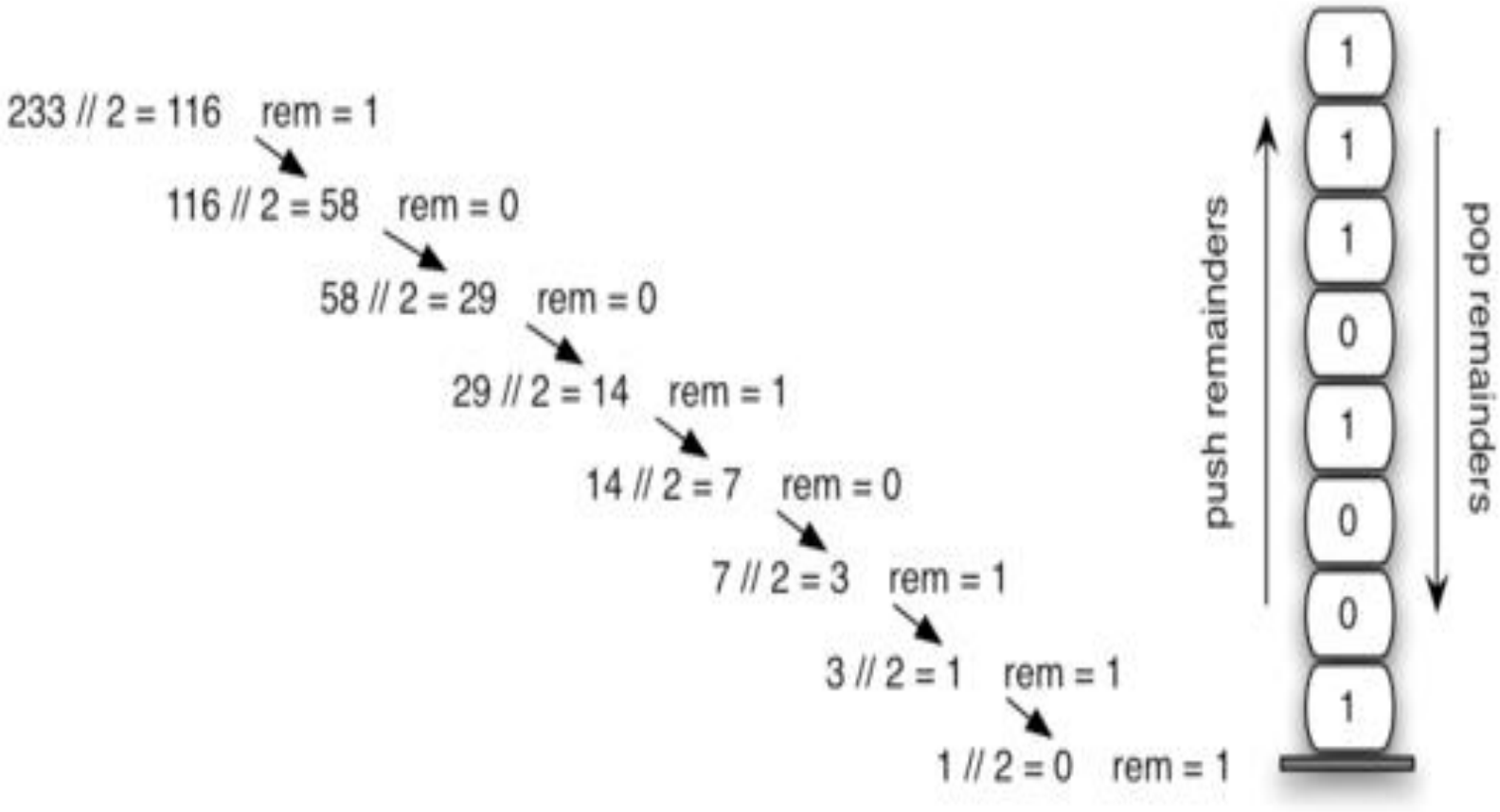
Converting Decimal to Binary: Consider the following pseudo code

1) Read (number)
2) Loop (number > 0)
    1) digit = number modulo 2
    2) print (digit)
    3) number = number / 2

The problem with this code is that it will print the binary number backwards. (ex: 19 becomes 11001instead of 10011.)

To remedy this problem, instead of printing the digit right away, we can push it onto the stack. Then after the number is done being converted, we pop the digit out of the stack and print it.

Converting Decimal Numbers to Binary Numbers



Finding Palindromes

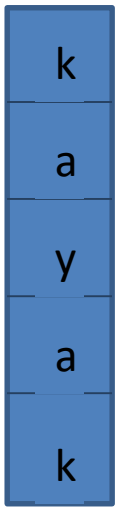
- Palindrome: a string that reads identically in either direction, letter by letter (ignoring case)
  - madam
  - kayak
  - "I saw I was I"
  - “Able was I ere I saw Elba”
  - "Level, madam, level"

Finding Palindromes

- Problem: Write a program that reads a string and determines whether it is a palindrome

Finding Palindromes (cont.)

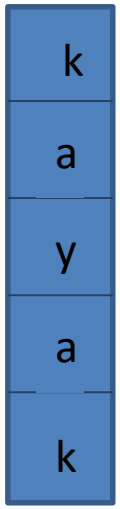
- Solving using a stack:
  - Push each string character, from left to right, onto a stack



k a y a k

Finding Palindromes (cont.)

- Solving using a stack:
  - Pop each character off the stack, appending each to the `StringBuilder result`



k a y a k

Testing

- To test this class using the following inputs:
  - a single character (always a palindrome)
  - multiple characters in a word
  - multiple words
  - different cases
  - even-length strings
  - odd-length strings
  - the empty string (considered a palindrome)

Implementation in C

```c
#include <stdio.h>

int MAXSIZE = 8;
int stack[8];
int top = -1;

int isempty() {

   if(top == -1)
      return 1;
   else
      return 0;
}
```

```
int isfull() {

   if(top == MAXSIZE)
      return 1;
   else
      return 0;
}

int peek() {
   return stack[top];
}
```

```
int pop() {
   int data;

   if(!isempty()) {
      data = stack[top];
      top = top - 1;
      return data;
   } else {
      printf("Could not retrieve data, Stack is empty.\n");
   }
}

int push(int data) {

   if(!isfull()) {
      top = top + 1;
      stack[top] = data;
   } else {
      printf("Could not insert data, Stack is full.\n");
   }
}
```

```
int main() {
   // push items on to the stack
   push(3);
   push(5);
   push(9);
   push(1);
   push(12);
   push(15);

   printf("Element at top of the stack: %d\n" ,peek());
   printf("Elements: \n");

   // print stack data
   while(!isempty()) {
      int data = pop();
      printf("%d\n",data);
   }

   printf("Stack full: %s\n" , isfull()?"true":"false");
   printf("Stack empty: %s\n" , isempty()?"true":"false");

   return 0;
}
```

The Towers of Hanoi:

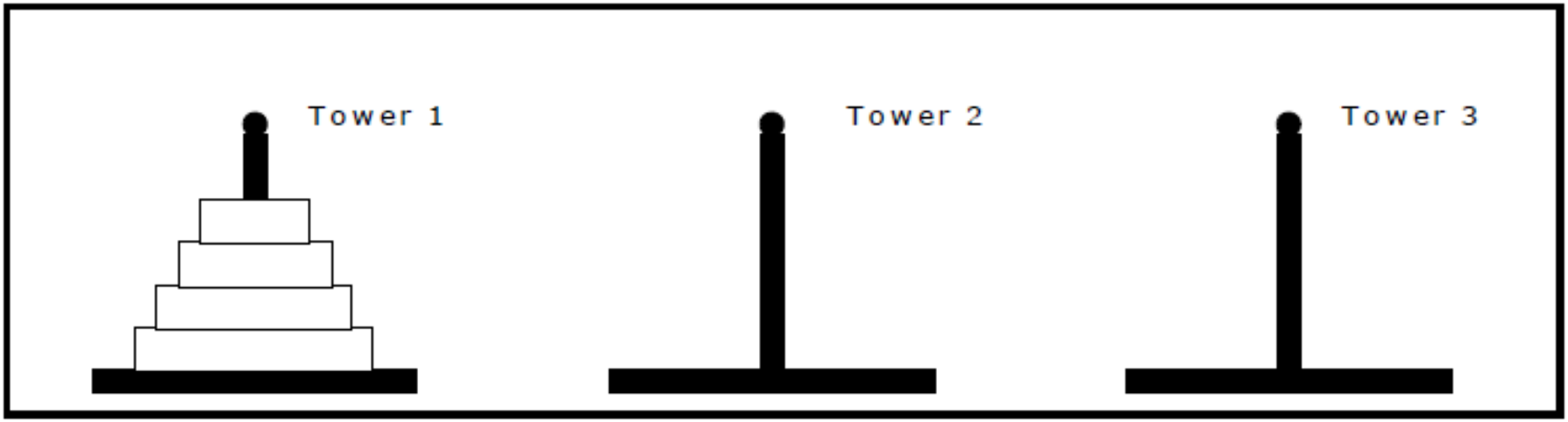
- In the game of Towers of Hanoi, there are three towers labeled 1, 2, and 3. The game starts with n disks on tower A.
- For simplicity, let n is 3. The disks are numbered from 1 to 3, and without loss of generality we may assume that the diameter of each disk is the same as its number. That is, disk 1 has diameter 1 (in some unit of measure), disk 2 has diameter 2, and disk 3 has diameter 3.
- All three disks start on tower A in the order 1, 2, 3.
- The objective of the game is to move all the disks in tower 1 to entire tower 3 using tower 2.
- That is, at no time can a larger disk be placed on a smaller disk.

The Towers of Hanoi: cont.…

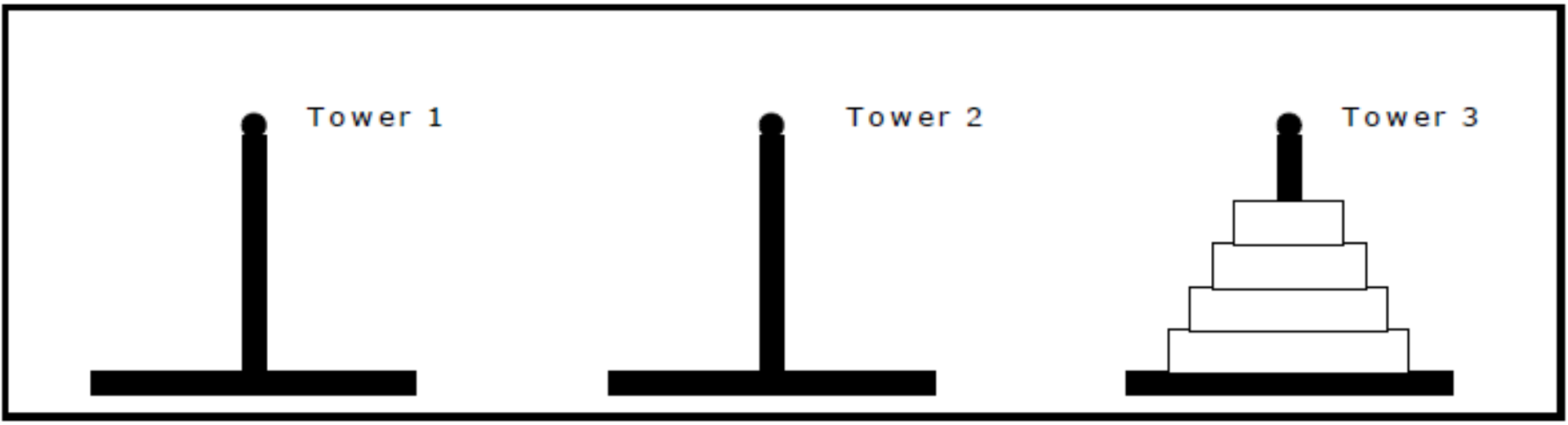
- The rules to be followed in moving the disks from tower 1 tower 3 using tower 2 are as follows:

1. Only one disk can be moved at a time.
2. Only the top disc on any tower can be moved to any other tower.
3. A larger disk cannot be placed on a smaller disk.

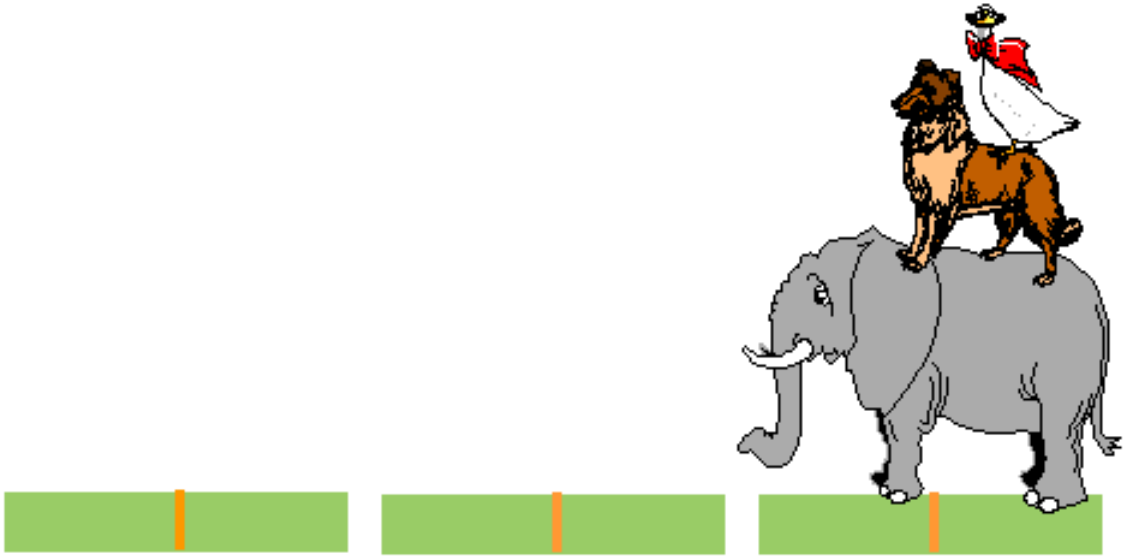
Initial set up of Towers of Hanoi



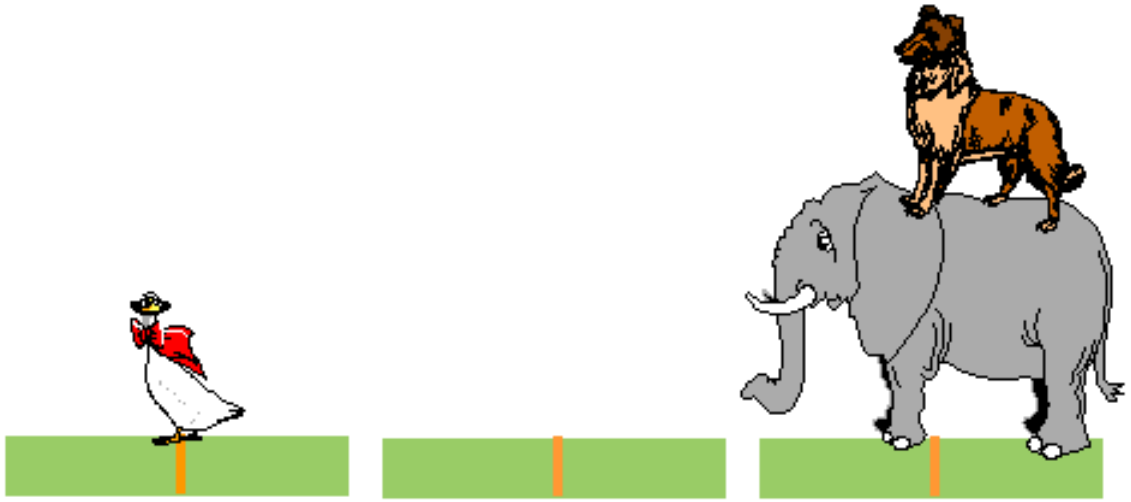
Final set up of Towers of Hanoi



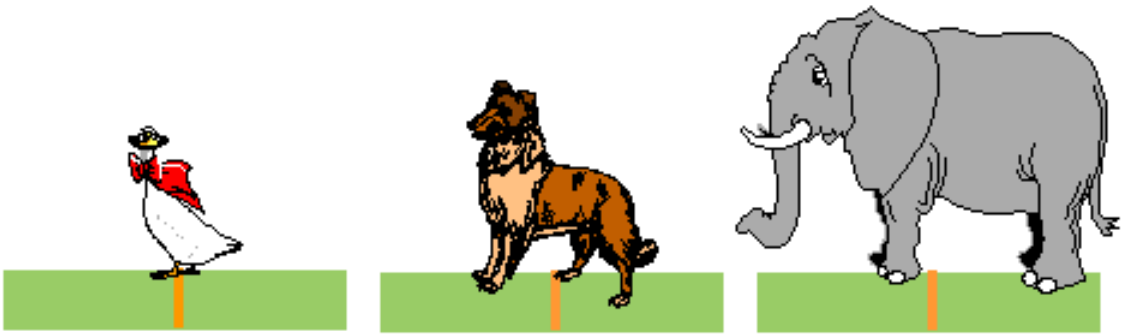
Towers of Hanoi



Towers of Hanoi



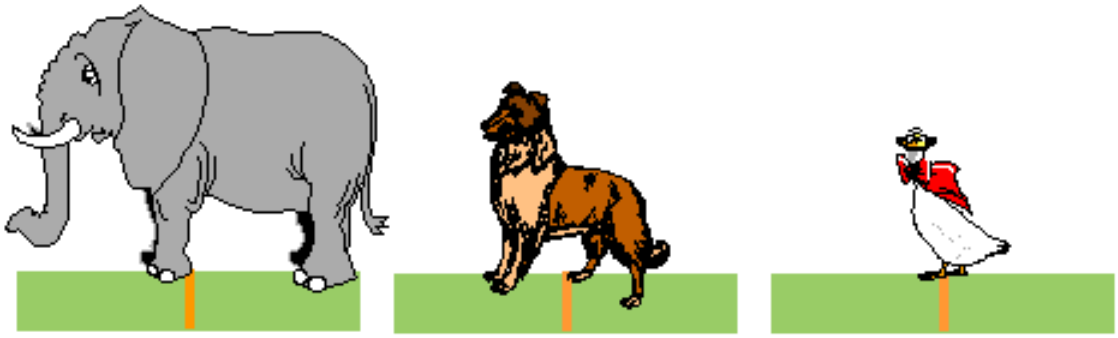
Towers of Hanoi



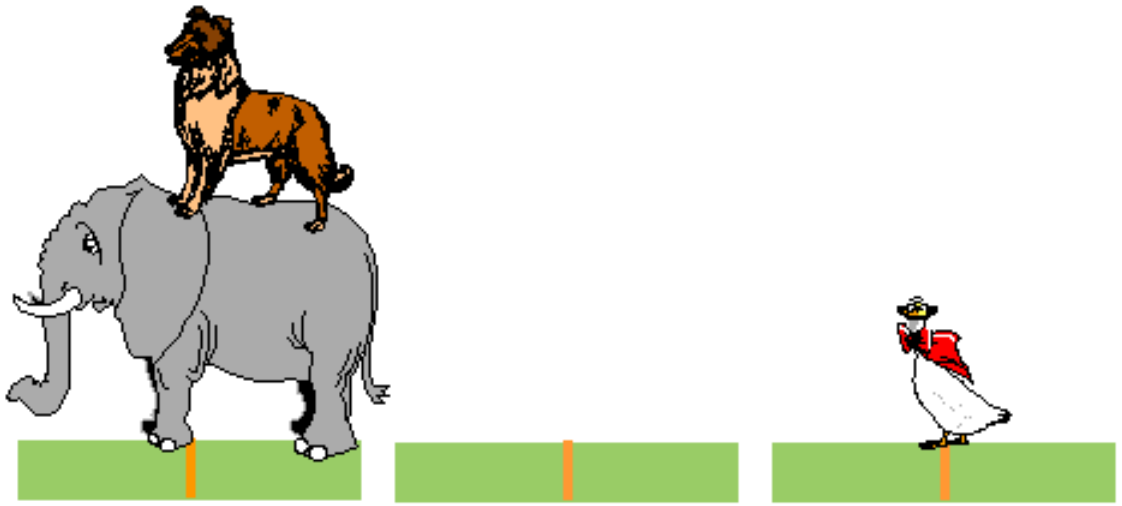
Towers of Hanoi

Towers of Hanoi

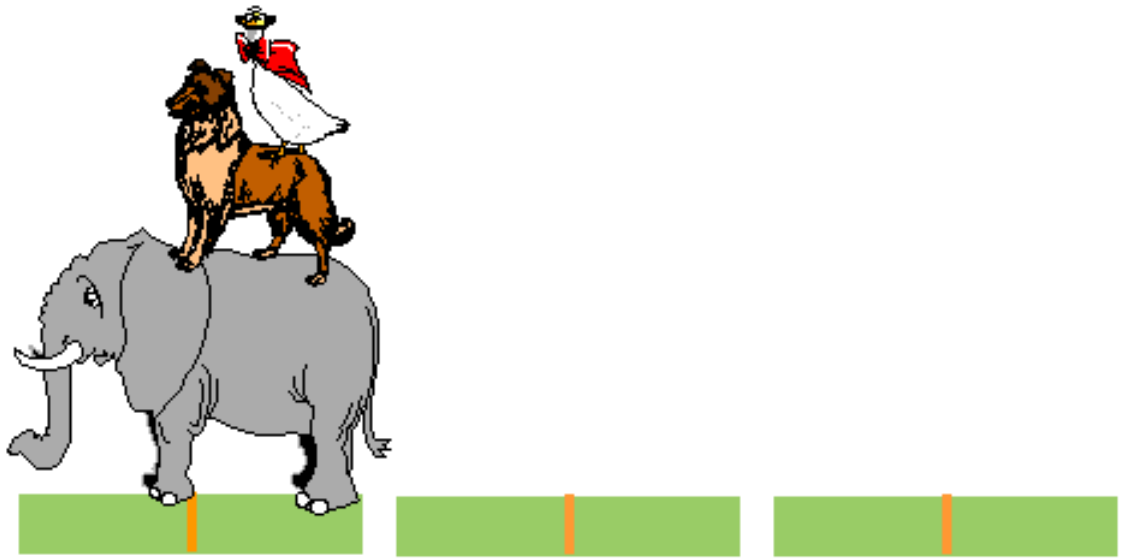
Towers of Hanoi



Towers of Hanoi



Towers of Hanoi



Towers of Hanoi – Recursive Solution

```
 void hanoi (int discs,
                    Stack fromPole,
                    Stack toPole,
                    Stack aux) {
Disc d;
if( discs >= 1) {
        hanoi(discs-1, fromPole, aux, toPole);
        d = fromPole.pop();
        toPole.push(d);
        hanoi(discs-1,aux, toPole, fromPole);
}
```

Towers of Hanoi- C program

```
#include <stdio.h>
#include <conio.h>
void towers_of_hanoi (int n, char *a, char *b, char *c);
int cnt=0;
int main (void)
{
    int n;
    printf("Enter number of discs: ");
    scanf("%d",&n);
    towers_of_hanoi (n, "Tower 1", "Tower 2", "Tower 3");
    getch();
}
```

Towers of Hanoi- C program cont..

```
void towers_of_hanoi (int n, char *a, char *b, char *c)
{
	if (n == 1)
	{
		++cnt;
		printf ("\n%5d: Move disk 1 from %s to %s", cnt, a, c);
		return;
	}
	else
	{
		towers_of_hanoi (n-1, a, c, b);
		++cnt;
		printf ("\n%5d: Move disk %d from %s to %s", cnt, n, a, c);
		towers_of_hanoi (n-1, b, a, c);
		return;
	}
}
```

Output of the program: Run 1

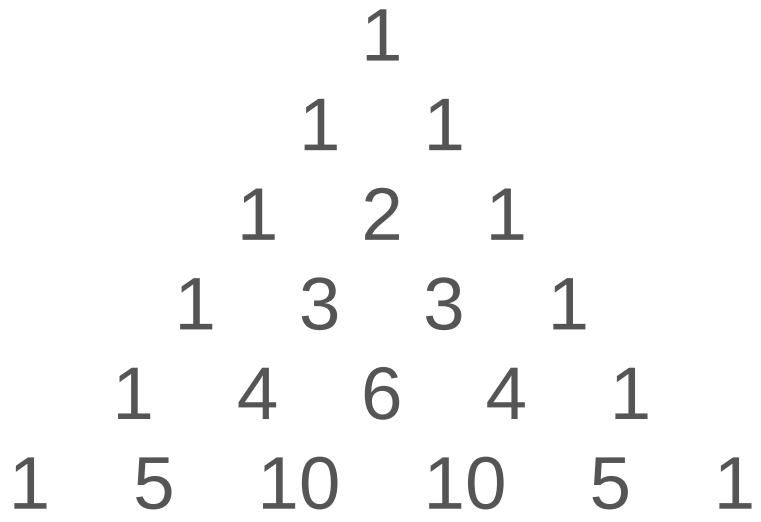
Enter the number of discs: 3
1: Move disk 1 from tower 1 to tower 3.
2: Move disk 2 from tower 1 to tower 2.
3: Move disk 1 from tower 3 to tower 2.
4: Move disk 3 from tower 1 to tower 3.
5: Move disk 1 from tower 2 to tower 1.
6: Move disk 2 from tower 2 to tower 3.
7: Move disk 1 from tower 1 to tower 3.

Output of the program: Run 2

Enter the number of discs: 4
1: Move disk 1 from tower 1 to tower 2.
2: Move disk 2 from tower 1 to tower 3.
3: Move disk 1 from tower 2 to tower 3.
4: Move disk 3 from tower 1 to tower 2.
5: Move disk 1 from tower 3 to tower 1.
6: Move disk 2 from tower 3 to tower 2.
7: Move disk 1 from tower 1 to tower 2.
8: Move disk 4 from tower 1 to tower 3.
9: Move disk 1 from tower 2 to tower 3.
10: Move disk 2 from tower 2 to tower 1.
11: Move disk 1 from tower 3 to tower 1.
12: Move disk 3 from tower 2 to tower 3.
13: Move disk 1 from tower 1 to tower 2.
14: Move disk 2 from tower 1 to tower 3.
15: Move disk 1 from tower 2 to tower 3.

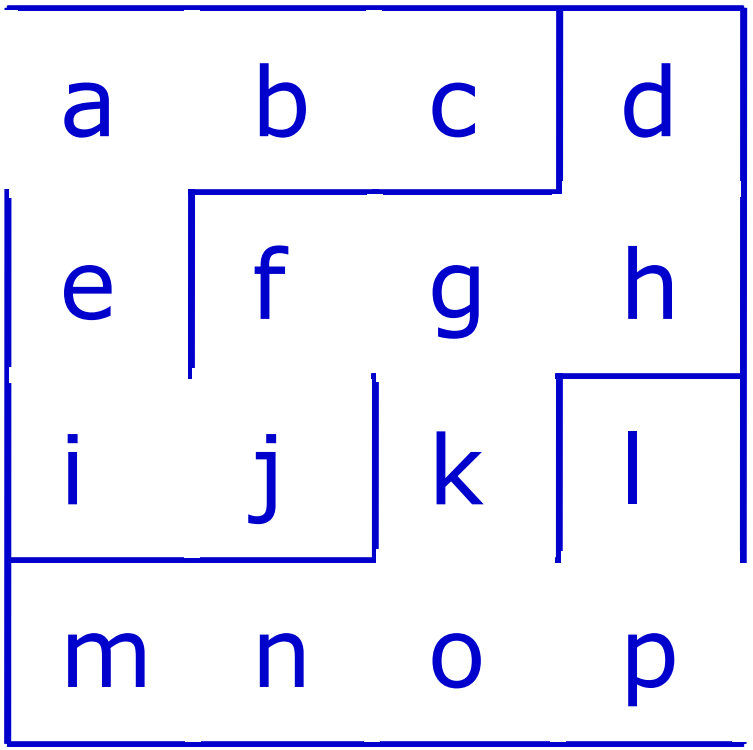
Exercise 2

Write a function which implements the Pascal's triangle:



Exercise 3 Stack Application - Maze

- Think about a grid of rooms separated by walls.
- Each room can be given a name.



Find a path thru the maze from a to p.

The towers of Hanoi Problem

- http://www.youtube.com/watch?v=aGlt2G-DC8c
- Animation + the towers of Hanoi
- http://www.cut-the-knot.org/recurrence/hanoi.shtml